NAMA	:	MUHAMMAD AL-FARISY
NIM	:	251012000019
KETERANGAN	:	UJI KOMPETENSI BNSP ASSOCIATE DATA SCIENCE

BAB 1 : Pendahuluan

1.1 Menentukan Objektif Bisnis

Permasalahan : Pengaruh dari kurang tepatnya dalam penentuan harga rumah ini berdampak pada penurunan penjualan agent properti yang menyebabkan harga tidak stabil dan customer tidak mampu membeli karena harganya yang naik secara drastis. Hal menjadi tantangan untuk agent properti untuk **menentukan harga rumah yang dijual secara tepat**, karena dalam menentukan harga rumah memiliki banyak faktor yang mendukung penentuan tinggi atau rendahnya harga rumah seperti lokasi rumah, umur rumah,jumlah ruangan,jumlah kamar tidur, keramaian lingkungan rumah, kawasan perumahan elit atau tidak dan jarak rumah dengan lokasi wisata. Tidak cukup hanya dengan membandingkan kondisi rumah yang akan dijual dengan rumah lainnya yang berada disekitar kemudian langsung menentukan harga rumah dengan harga pasar atau langsung menentukan hanya mengikuti harga pasar.

Objektif bisnis : dari proyek ini adalah membantu agen properti menentukan harga rumah yang lebih akurat dan kompetitif dengan memanfaatkan analisis data dan machine learning.

Secara spesifik, tujuan bisnis yang ingin dicapai adalah:

- Membantu agen properti menentukan harga rumah yang sesuai dengan kondisi pasar sehingga tidak terlalu tinggi maupun terlalu rendah.
- Meningkatkan peluang penjualan rumah dengan harga yang realistis dan dapat diterima oleh calon pembeli.
- Mengurangi risiko rumah tidak terjual dalam waktu lama akibat harga yang tidak sesuai.
- Membantu agen properti memberikan rekomendasi harga berbasis data kepada pemilik rumah.
- Meningkatkan kepercayaan client terhadap agen properti karena keputusan harga didasarkan pada analisis data yang objektif.

1.2 Membuat daftar terminologi, sumber daya, kebutuhan, asumsi dan Batasan

Terminologi :

Nama Kolom	Deskripsi Kolom
longitude	Ukuran seberapa jauh ke barat sebuah rumah; nilai yang lebih tinggi lebih jauh ke barat
latitude	Ukuran seberapa jauh ke utara sebuah rumah;

Nama Kolom	Deskripsi Kolom
	nilai yang lebih tinggi lebih jauh ke utara
housing_median_age	Usia rata-rata sebuah rumah dalam satu blok (angka yang lebih rendah adalah bangunan yang lebih baru
total_rooms	Jumlah total ruangan dalam rumah didalam satu blok
total_bedrooms	Jumlah total kamar tidur dalam satu blok
population	Jumlah total orang yang tinggal dalam satu blok
households	Jumlah total rumah tangga / sekelompok orang yang tinggal dalam satu unit rumah di satu blok
median_income	Pendapatan rata-rata untuk rumah tangga dalam satu blok rumah (diukur dalam puluhan ribu Dolar AS)
ocean_proximity	Lokasi rumah dengan laut
median_house_value	Nilai median rumah untuk rumah tangga dalam satu blok (diukur dalam Dolar AS)

Sumber Daya :

```
- Data :

    - Dataset California Housing Dataset

    - Dataset berisi informasi geografis, demografi, dan karakteristik
rumah yang mempengaruhi harga rumah.

- Manusia :

    - 1 Data Scientist yang melakukan analisis data dan pembangunan
model

    - Subject Matter Expert dari bidang properti atau real estate
untuk memberikan pemahaman bisnis

- Tools :

    - Python

    - Visual Code

- Library Python

    - Dalam proses analisis data dan pembangunan model machine
learning, proyek ini menggunakan beberapa library Python sebagai
berikut:
```

```
    - Pandas :
    Digunakan untuk melakukan manipulasi dan analisis data dalam
bentuk struktur data DataFrame.

    - NumPy :
    Digunakan untuk operasi numerik dan pengolahan array multidimensi.

    - Matplotlib :
    Digunakan untuk membuat visualisasi data seperti grafik dan plot.

    - Seaborn :
    Digunakan untuk membuat visualisasi statistik yang lebih
informatif dan menarik.

    - Scikit-Learn :
    Digunakan untuk membangun model machine learning serta melakukan
proses preprocessing data seperti:

        - train-test split :

        - cross validation

        - pipeline

        - model evaluation

    - Category Encoders
    Digunakan untuk melakukan encoding pada variabel kategorikal agar
dapat diproses oleh model machine learning.

    - Algoritma Machine Learning yang Digunakan : Regresi Linear
    - Evaluation Metrics :
        - Mean Absolute Error (MAE) : Mengukur rata-rata selisih
absolut antara nilai prediksi dan nilai aktual.
        - Mean Absolute Percentage Error (MAPE) : Mengukur kesalahan
prediksi dalam bentuk persentase terhadap nilai aktual.
```

Asumsi :

- Data historis harga rumah dapat digunakan sebagai dasar untuk memprediksi harga rumah di masa depan.
- Variabel seperti pendapatan rumah tangga, jumlah rumah tangga, dan lokasi geografis memiliki pengaruh terhadap harga rumah.
- Data yang digunakan cukup representatif terhadap kondisi pasar properti di California.

Batasan :

- Dataset hanya mencakup wilayah California, sehingga model tidak dapat langsung digunakan untuk wilayah lain.

- Dataset tidak mencakup beberapa faktor yang mempengaruhi harga rumah seperti kondisi interior rumah, fasilitas rumah, dan desain bangunan.
- Data yang digunakan merupakan data agregasi per blok wilayah, bukan data detail per rumah.

1.3 Mengidentifikasi risiko dan alternatif pelaksanaan

Risiko	Deskripsi	Mitigasi / Alternatif
Kualitas Data	Dataset kemungkinan memiliki nilai yang hilang (missing value) atau data yang tidak konsisten, seperti pada variabel *total_bedrooms*. Hal ini dapat mempengaruhi proses analisis dan hasil model.	Melakukan proses **data cleaning** dengan menangani missing value menggunakan teknik imputasi seperti **median atau mean** agar data tetap dapat digunakan dalam proses modeling.
Outlier Data	Beberapa data rumah mungkin memiliki nilai yang sangat ekstrem sehingga dapat mempengaruhi performa model dalam memprediksi harga rumah.	Melakukan **analisis outlier** menggunakan visualisasi seperti boxplot dan mempertimbangkan proses **transformasi data atau filtering outlier** yang tidak wajar.
Model Overfitting	Model machine learning mungkin terlalu menyesuaikan data training sehingga performanya menjadi kurang baik ketika digunakan pada data baru.	Menggunakan teknik **cross validation**, melakukan **hyperparameter tuning**, serta membandingkan beberapa algoritma untuk mendapatkan model yang lebih general.
Perubahan Kondisi Pasar Properti	Harga rumah dapat berubah seiring waktu karena faktor ekonomi, inflasi, atau kebijakan pemerintah yang tidak tercermin dalam dataset historis.	Melakukan **pembaruan model secara berkala** dengan data terbaru agar model tetap relevan dengan kondisi pasar properti.
Keterbatasan Variabel Dataset	Dataset mungkin belum mencakup semua faktor yang mempengaruhi harga rumah seperti fasilitas umum, kondisi lingkungan, atau akses transportasi.	Menggunakan dataset tambahan di masa depan atau melakukan **feature engineering** untuk meningkatkan kemampuan model dalam memprediksi harga rumah.

1.4 Menentukan biaya dan keuntungan

Analisis Biaya dan Keuntungan

• **Estimasi Biaya Proyek**

Pengembangan model machine learning untuk prediksi harga rumah membutuhkan infrastruktur komputasi dan penyimpanan data. Estimasi biaya dihitung menggunakan layanan cloud seperti Google Cloud Platform.

Komponen biaya antara lain:

- Compute Engine (VM instance untuk training model) ≈ **$100 per bulan**
- Cloud Storage untuk penyimpanan dataset ≈ **$20 per bulan**

Perhitungan biaya operasional sistem:

Biaya per bulan

$100 + $20 = **$120 per bulan**

Biaya per tahun

$120 × 12 = **$1,440 per tahun**

Dengan demikian, estimasi biaya operasional sistem machine learning untuk mendukung proses prediksi harga rumah adalah sekitar **$1,440 per tahun**.

Referensi harga:
https://cloud.google.com/pricing

• **Potensi Keuntungan**

Dalam bisnis properti, penentuan harga rumah yang tidak tepat dapat menyebabkan dua kondisi utama, yaitu **underprice** dan **overprice**.

- **Underprice (harga terlalu rendah)** dapat menyebabkan rumah dijual di bawah nilai pasar sehingga mengurangi potensi keuntungan penjualan.
- **Overprice (harga terlalu tinggi)** dapat menyebabkan rumah kurang kompetitif di pasar dan membutuhkan waktu lebih lama untuk terjual.

Dengan memanfaatkan model machine learning untuk memprediksi harga rumah berdasarkan faktor geografis, demografis, dan karakteristik properti, perusahaan dapat memperoleh rekomendasi harga yang lebih mendekati nilai pasar.

Strategi penyesuaian harga yang dapat diterapkan antara lain:

- Jika harga rumah terindikasi **lebih rendah dari nilai pasar (underprice)**, maka harga dapat dinaikkan sekitar **5%** untuk mengoptimalkan keuntungan.
- Jika harga rumah terindikasi **lebih tinggi dari nilai pasar (overprice)**, maka harga dapat diturunkan sekitar **2–3%** agar lebih kompetitif di pasar.

Sebagai ilustrasi, jika rata-rata harga rumah berada pada kisaran **$200,000**, maka peningkatan harga sebesar **5%** dapat memberikan tambahan nilai sekitar **$10,000 per rumah**.

Jika strategi ini diterapkan pada **50 rumah dalam satu tahun**, maka potensi peningkatan nilai transaksi dapat mencapai sekitar:

$10,000 × 50 = $500,000

Kesimpulan

Dengan estimasi biaya operasional sistem sekitar **$1,440 per tahun**, serta potensi peningkatan nilai transaksi yang dapat mencapai ratusan ribu dolar dari optimalisasi harga rumah, penerapan sistem machine learning untuk prediksi harga rumah dinilai **layak secara bisnis dan memberikan manfaat yang signifikan** dalam mendukung pengambilan keputusan penentuan harga properti.

1.5 Menentukan Tujuan Teknis Data Science

• Menyusun Tujuan Teknis Data Science

Tujuan teknis dari proyek ini adalah membangun dan membandingkan beberapa model machine learning untuk memprediksi harga rumah. Model yang digunakan antara lain:

- **Linear Regression (model_lr)**
- **K-Nearest Neighbors Regressor (model_knn)**
- **Decision Tree Regressor (model_dt)**
- **Random Forest Regressor (model_rf)**
- **XGBoost Regressor (model_xgb)**

Dengan membangun berbagai model, diharapkan dapat menemukan model dengan performa terbaik untuk memprediksi **median_house_value** berdasarkan fitur-fitur seperti **longitude, latitude, housing_median_age, total_rooms, total_bedrooms, population, households, median_income**, dan **ocean_proximity**.

• Menentukan Kriteria Kesuksesan Tujuan Teknis

Keberhasilan model akan diukur menggunakan metrik evaluasi regresi berikut:

1. **Mean Absolute Error (MAE)**
   Mengukur rata-rata selisih absolut antara harga rumah aktual dan prediksi model. Semakin kecil MAE, semakin akurat model.
2. **Mean Absolute Percentage Error (MAPE)**
   Mengukur kesalahan prediksi dalam bentuk persentase, memudahkan interpretasi performa model relatif terhadap harga rumah aktual.

Kriteria keberhasilan proyek:

- Model terbaik memiliki **MAE rendah**, mendekati nilai optimal untuk dataset yang digunakan.

- Model terbaik memiliki **MAPE < 20%**, sehingga prediksi harga rumah relatif akurat terhadap harga pasar.

Dengan kriteria ini, proyek data science dapat memastikan prediksi harga rumah yang akurat dan dapat digunakan untuk mendukung pengambilan keputusan bisnis yang lebih baik.

BAB 2 : Metedologi

2.1 Menelaah Data

- Menganalisis tipe dan relasi data

```
## Import library yang dibutuhkan untuk eksplorasi dataset
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

import warnings
warnings.filterwarnings('ignore')

df = pd.read_csv(r'D:\Big Data\Pelatihan BNSP UNPAM\Note\
data_california_house.csv')
df.tail(50)
```

```
       longitude  latitude  housing_median_age  total_rooms
total_bedrooms  \
14398    -118.59     34.25                15.0       9716.0
2387.0
14399    -119.08     36.13                21.0       2271.0
376.0
14400    -117.76     34.06                30.0       1700.0
504.0
14401    -117.66     33.64                17.0       3173.0
501.0
14402    -121.90     37.30                39.0       3627.0
666.0
14403    -117.92     33.84                45.0       2019.0
394.0
14404    -120.06     39.25                21.0       2459.0
525.0
14405    -121.90     36.93                22.0       7281.0
1233.0
14406    -122.25     37.46                33.0       6841.0
950.0
14407    -118.32     33.83                19.0       3792.0
790.0
14408    -122.67     38.24                29.0       2644.0
```

```
464.0
14409    -121.91     39.14                45.0        845.0
155.0
14410    -121.34     37.99                14.0       3111.0
498.0
14411    -117.31     33.00                30.0       1631.0
310.0
14412    -119.67     36.33                19.0       1241.0
244.0
14413    -122.28     37.49                25.0       7335.0
1157.0
14414    -122.08     37.63                33.0        691.0
127.0
14415    -122.04     37.67                18.0       3000.0
419.0
14416    -117.94     33.87                46.0       2066.0
450.0
14417    -117.05     33.05                 6.0       7916.0
1293.0
14418    -120.41     36.77                24.0       1335.0
312.0
14419    -118.25     33.89                36.0        406.0
71.0
14420    -119.95     38.94                24.0       2180.0
517.0
14421    -121.87     37.26                24.0       2383.0
343.0
14422    -118.15     33.87                29.0       2690.0
659.0
14423    -118.30     34.03                40.0       1695.0
374.0
14424    -118.31     33.98                52.0       1837.0
426.0
14425    -119.82     36.77                41.0       1441.0
274.0
14426    -117.73     34.06                34.0        344.0
108.0
14427    -118.03     34.01                10.0       6531.0
1036.0
14428    -117.92     34.06                34.0       2819.0
609.0
14429    -122.33     37.57                27.0       3085.0
876.0
14430    -119.01     35.38                44.0        434.0
110.0
14431    -122.13     37.15                39.0       2854.0
613.0
14432    -117.77     33.85                13.0       5415.0
827.0
```

```
14433    -121.99     38.27                16.0       4138.0
758.0
14434    -122.22     37.47                35.0        367.0
113.0
14435    -117.20     32.77                31.0       1952.0
471.0
14436    -119.21     36.38                18.0       2158.0
413.0
14437    -118.07     34.12                43.0       1554.0
287.0
14438    -122.43     37.75                52.0       2285.0
509.0
14439    -117.24     33.20                25.0       1631.0
415.0
14440    -117.00     32.64                11.0       3098.0
490.0
14441    -117.10     32.73                24.0       2927.0
704.0
14442    -120.06     36.94                19.0        901.0
183.0
14443    -121.26     38.27                20.0       1314.0
229.0
14444    -120.89     37.48                27.0       1118.0
195.0
14445    -121.90     36.58                31.0       1431.0
NaN
14446    -117.93     33.62                34.0       2125.0
498.0
14447    -115.56     32.80                15.0       1171.0
328.0

       population  households  median_income ocean_proximity  \
14398      4992.0      2225.0         3.6231       <1H OCEAN
14399      1145.0       372.0         3.1528          INLAND
14400      1719.0       459.0         2.2270          INLAND
14401      1555.0       520.0         6.7079       <1H OCEAN
14402      1531.0       635.0         4.5370       <1H OCEAN
14403      1549.0       377.0         4.6111       <1H OCEAN
14404       584.0       233.0         3.0100          INLAND
14405      1849.0       832.0         5.3276      NEAR OCEAN
14406      2681.0       980.0         7.1088      NEAR OCEAN
14407      2105.0       834.0         5.2363       <1H OCEAN
14408      1372.0       450.0         5.0544       <1H OCEAN
14409       343.0       136.0         2.1250          INLAND
14410      1178.0       525.0         6.5560          INLAND
14411       665.0       297.0         6.8443      NEAR OCEAN
14412       850.0       237.0         2.9375          INLAND
14413      2626.0      1049.0         6.5475      NEAR OCEAN
14414       431.0       149.0         4.2500        NEAR BAY
```

14415	1155.0	415.0	6.8233	NEAR BAY
14416	1275.0	448.0	3.9375	<1H OCEAN
14417	2741.0	1204.0	5.6436	<1H OCEAN
14418	1180.0	267.0	1.9470	INLAND
14419	268.0	77.0	3.9000	<1H OCEAN
14420	755.0	223.0	2.5875	INLAND
14421	1146.0	341.0	5.6223	<1H OCEAN
14422	1747.0	617.0	3.3713	<1H OCEAN
14423	1138.0	357.0	2.7125	<1H OCEAN
14424	1062.0	343.0	2.0000	<1H OCEAN
14425	646.0	296.0	3.0568	INLAND
14426	315.0	119.0	3.1786	INLAND
14427	2975.0	1018.0	6.2319	<1H OCEAN
14428	1718.0	558.0	3.5547	<1H OCEAN
14429	1453.0	896.0	3.4333	NEAR OCEAN
14430	274.0	86.0	1.1944	INLAND
14431	1338.0	518.0	3.9423	NEAR OCEAN
14432	2061.0	714.0	7.3681	<1H OCEAN
14433	1762.0	723.0	3.1979	INLAND
14434	398.0	109.0	2.5000	NEAR OCEAN
14435	936.0	462.0	2.8621	NEAR OCEAN
14436	1461.0	395.0	2.0216	INLAND
14437	802.0	277.0	4.2312	INLAND
14438	839.0	456.0	4.7946	NEAR BAY
14439	1045.0	386.0	2.4505	<1H OCEAN
14440	1391.0	484.0	4.9792	NEAR OCEAN
14441	2005.0	668.0	2.2375	NEAR OCEAN
14442	700.0	190.0	2.2375	INLAND
14443	712.0	219.0	4.4125	INLAND
14444	647.0	209.0	2.9135	INLAND
14445	704.0	393.0	3.1977	NEAR OCEAN
14446	1052.0	468.0	5.6315	<1H OCEAN
14447	1024.0	298.0	1.3882	INLAND

	median_house_value
14398	193300.0
14399	113700.0
14400	91900.0
14401	250800.0
14402	345900.0
14403	223000.0
14404	163500.0
14405	335500.0
14406	443300.0
14407	310000.0
14408	261800.0
14409	62000.0
14410	234700.0
14411	492500.0

```
14412              81700.0
14413             500001.0
14414             192600.0
14415             332600.0
14416             187000.0
14417             278600.0
14418              68900.0
14419             115800.0
14420             173400.0
14421             265700.0
14422             198200.0
14423             150000.0
14424              96500.0
14425              71300.0
14426             117800.0
14427             403700.0
14428             197600.0
14429             290000.0
14430              57500.0
14431             180300.0
14432             353100.0
14433             137500.0
14434             166700.0
14435             196900.0
14436              58000.0
14437             272600.0
14438             355600.0
14439             147500.0
14440             170400.0
14441             102900.0
14442              64300.0
14443             144600.0
14444             159400.0
14445             289300.0
14446             484600.0
14447              69400.0
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14448 entries, 0 to 14447
Data columns (total 10 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   longitude           14448 non-null  float64
 1   latitude            14448 non-null  float64
 2   housing_median_age  14448 non-null  float64
 3   total_rooms         14448 non-null  float64
 4   total_bedrooms      14311 non-null  float64
 5   population          14448 non-null  float64
```

```
 6   households          14448 non-null  float64
 7   median_income       14448 non-null  float64
 8   ocean_proximity     14448 non-null  object
 9   median_house_value  14448 non-null  float64
dtypes: float64(9), object(1)
memory usage: 1.1+ MB
```

- Terdapat 2 tipe data yaitu :
  - **Tipe data float berisikan kolom** : *longtitude,latitude,housing_median_age,total_rooms,total_bedrooms,populations,households,median_income,median_house_values*
  - **Tipe data Object berisikan kolom :** *ocean_proximity*. Namun untuk tipe kolom housing_median_age,total_rooms,total_bedrooms,population,households akan diubah menjadi tipe data integer dengan alasan untuk nama nama kolom kolom housing_median_age,total_rooms,total_bedrooms,population,households ini termasuk dalam bilangan bulat.
- Bagian yang menjadi target variabel Y adalah : **median_house_value**

```
df.shape
```

```
(14448, 10)
```

- Terdapat 14.448 baris dengan 10 kolom data
- Attributes (Kolom) dan Records (Baris) Data

Nama Kolom	Deskripsi Kolom
longitude	Ukuran seberapa jauh ke barat sebuah rumah; nilai yang lebih tinggi lebih jauh ke barat
latitude	Ukuran seberapa jauh ke utara sebuah rumah; nilai yang lebih tinggi lebih jauh ke utara
housing_median_age	Usia rata-rata sebuah rumah dalam satu blok (angka yang lebih rendah adalah bangunan yang lebih baru)
total_rooms	Jumlah total ruangan dalam rumah di dalam satu blok
total_bedrooms	Jumlah total kamar tidur dalam satu blok
population	Jumlah total orang yang tinggal dalam satu blok
households	Jumlah total rumah tangga / sekelompok orang yang tinggal dalam satu unit rumah di satu blok
median_income	Pendapatan rata-rata untuk rumah tangga dalam satu blok rumah (diukur dalam puluhan ribu Dolar AS)
ocean_proximity	Lokasi rumah relatif terhadap laut
median_house_value	Nilai median rumah untuk rumah tangga dalam satu blok (diukur dalam Dolar AS)

2.2 Memvalidasi Data

- Melakukan pengecekan kelengkapan data

- Membuat rekomendasi kelengkapan data

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14448 entries, 0 to 14447
Data columns (total 10 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   longitude           14448 non-null  float64
 1   latitude            14448 non-null  float64
 2   housing_median_age  14448 non-null  float64
 3   total_rooms         14448 non-null  float64
 4   total_bedrooms      14311 non-null  float64
 5   population          14448 non-null  float64
 6   households          14448 non-null  float64
 7   median_income       14448 non-null  float64
 8   ocean_proximity     14448 non-null  object
 9   median_house_value  14448 non-null  float64
dtypes: float64(9), object(1)
memory usage: 1.1+ MB
```

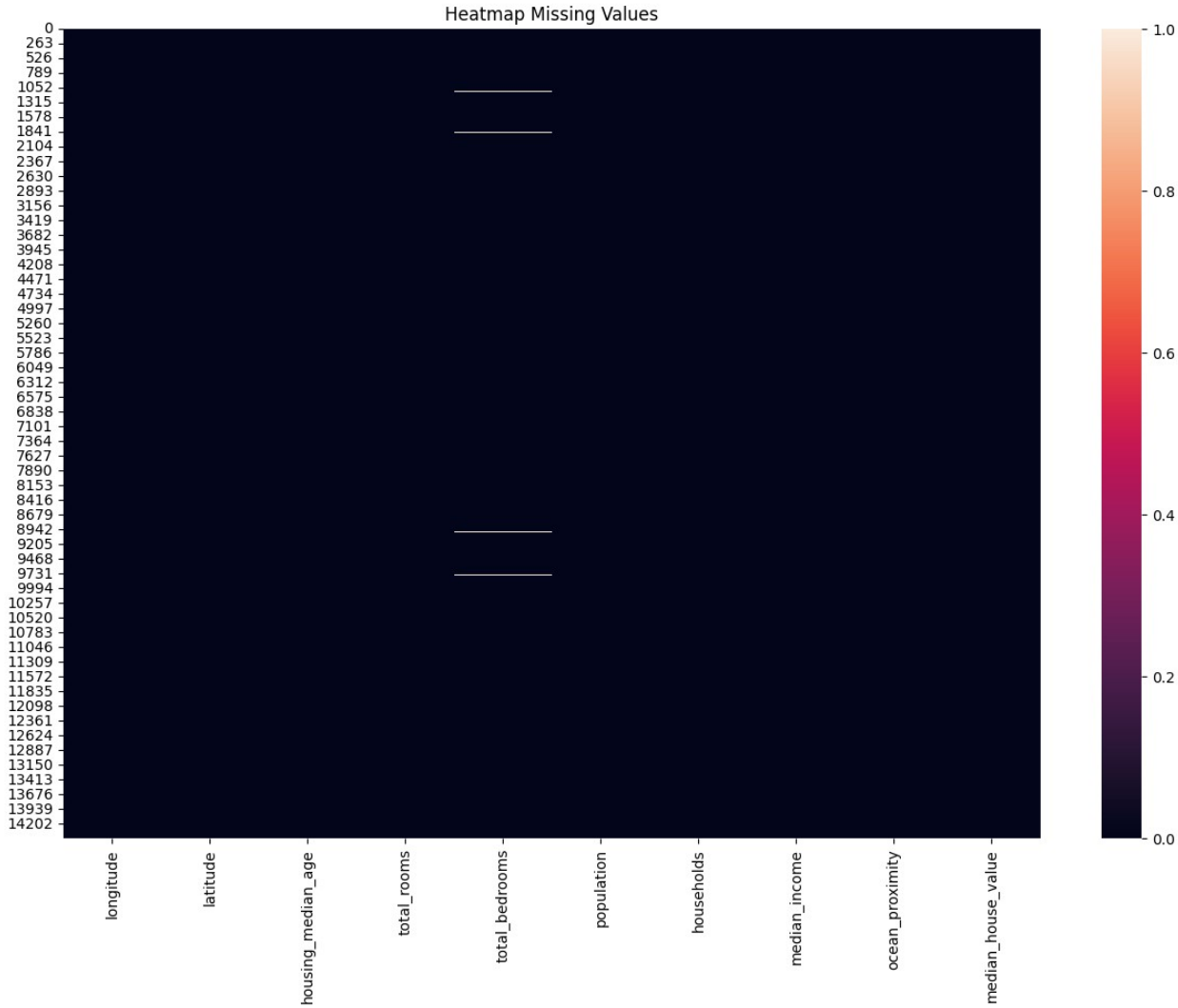
- Terdapat tipe data yang seharusnya menggunakan type data **int** namun menggunakan **float** :
  - total_rooms
  - total_bedrooms
  - population
  - households

```
print(f"Jumlah data duplikat: {df.duplicated().sum()}")
```

```
Jumlah data duplikat: 0
```

Dari hasil pengecekan duplikat, dapat dilihat hasilnya adalah 0 yang artinya tidak terdapat data duplikat.

```
plt.figure(figsize=(15,10))
sns.heatmap(df.isna(),cbar='YlOrRd_r')
plt.title('Heatmap Missing Values')
plt.show()
```




```
print(f"Jumlah data yang memiliki missing values:
{df.isna().any(axis=1).sum()}\n")
print(f"Data yang memiliki missing values ada di kolom :")
df.isna().sum()
```

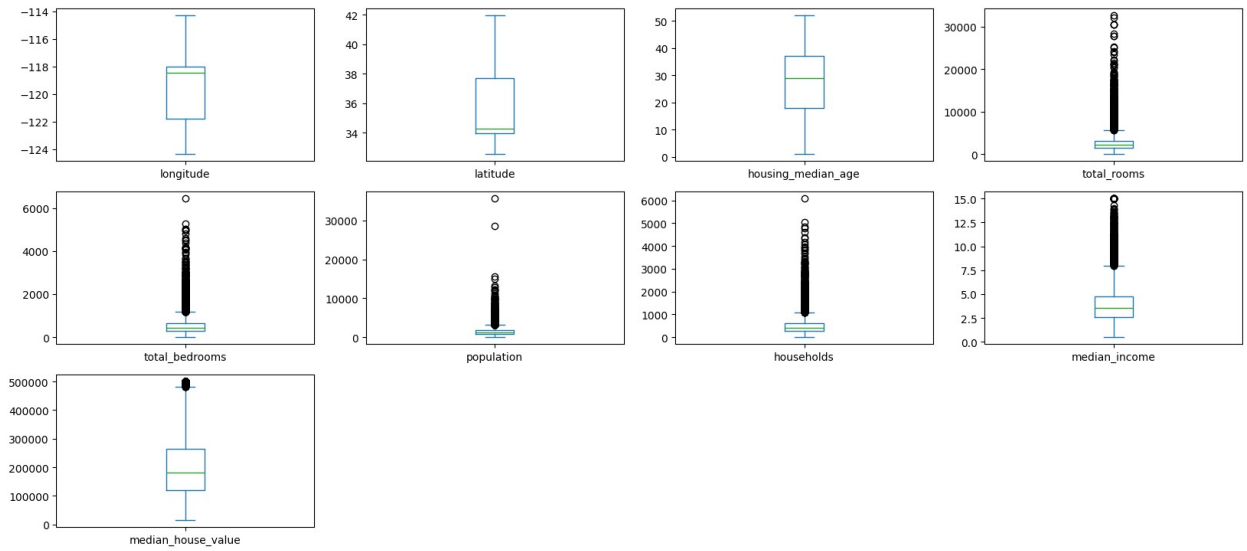
```
Jumlah data yang memiliki missing values: 137

Data yang memiliki missing values ada di kolom :

longitude               0
latitude                0
housing_median_age      0
total_rooms             0
total_bedrooms        137
population              0
households              0
median_income           0
ocean_proximity         0
```

```
median_house_value      0
dtype: int64
```

Dari visualisasi di atas terdapat missing values pada kolom total_bedrooms, perlu dilakukan perhitungan berapa jumlah baris yang memiliki missing values dari kolom total_bedrooms.

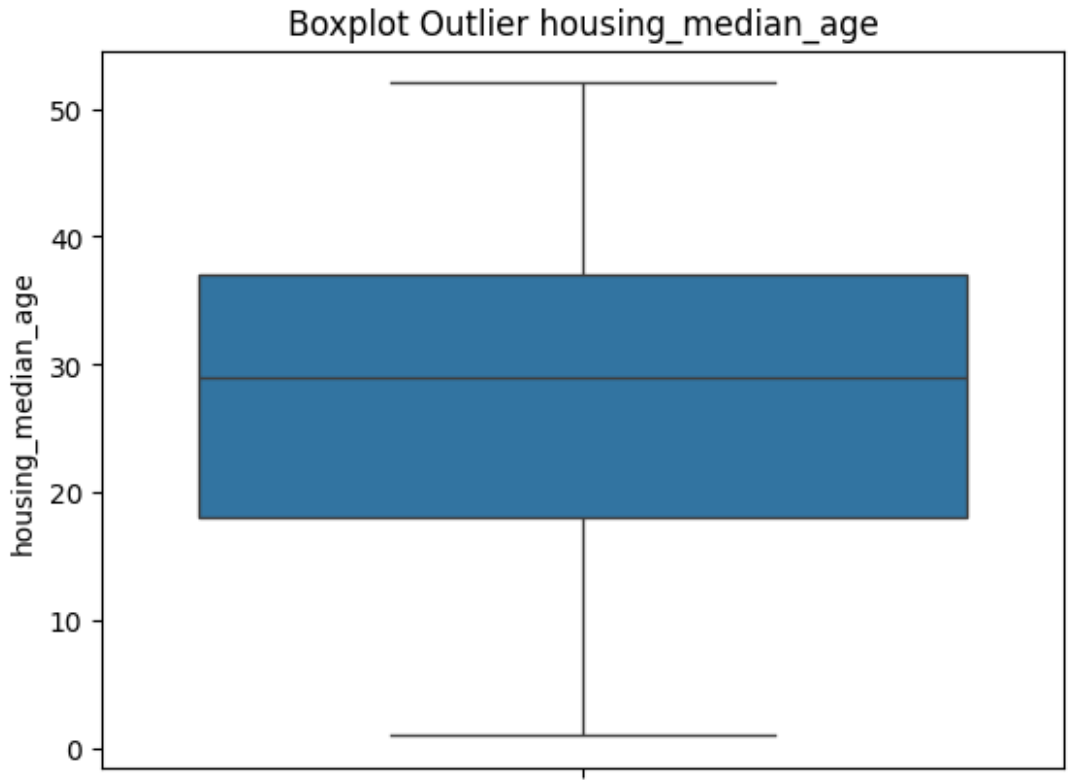
Dari total baris 14.448 **terdapat missing values sebanyak 137 baris atau setara 0.948 %**, jumlah missing values ini **tidak mencapai 1 persen dari keseluruhan baris**. Jika missing values di hapus maka **sisa total baris 14.331 atau setara 99.052 %**. Dalam hal ini missing values akan di hapus karena memiliki jumlah sedikit dan data aktual yang hilang juga sedikit sehingga tidak berpengaruh signifikan pada model machine learning.

```
# Plot boxplot untuk mengecek outliers
df.plot(kind='box', figsize=(20,15), subplots=True, layout=(5,4))
plt.show()
```




```
def check_outliers(df,col):
    Q1=df[col].describe()['25%']
    Q3=df[col].describe()['75%']
    IQR=Q3-Q1
    Upper_fence=Q3+(1.5*IQR)
    Lower_fence=Q1-(1.5*IQR)
    df_outliers=df[(df[col]<Lower_fence) | (df[col]>Upper_fence)]
    if len(df_outliers)==0:
        return 'Tidak Ada Outliers'
    else:
        return df_outliers

sns.boxplot(df['housing_median_age'])
plt.title('Boxplot Outlier housing_median_age')
plt.show()
```

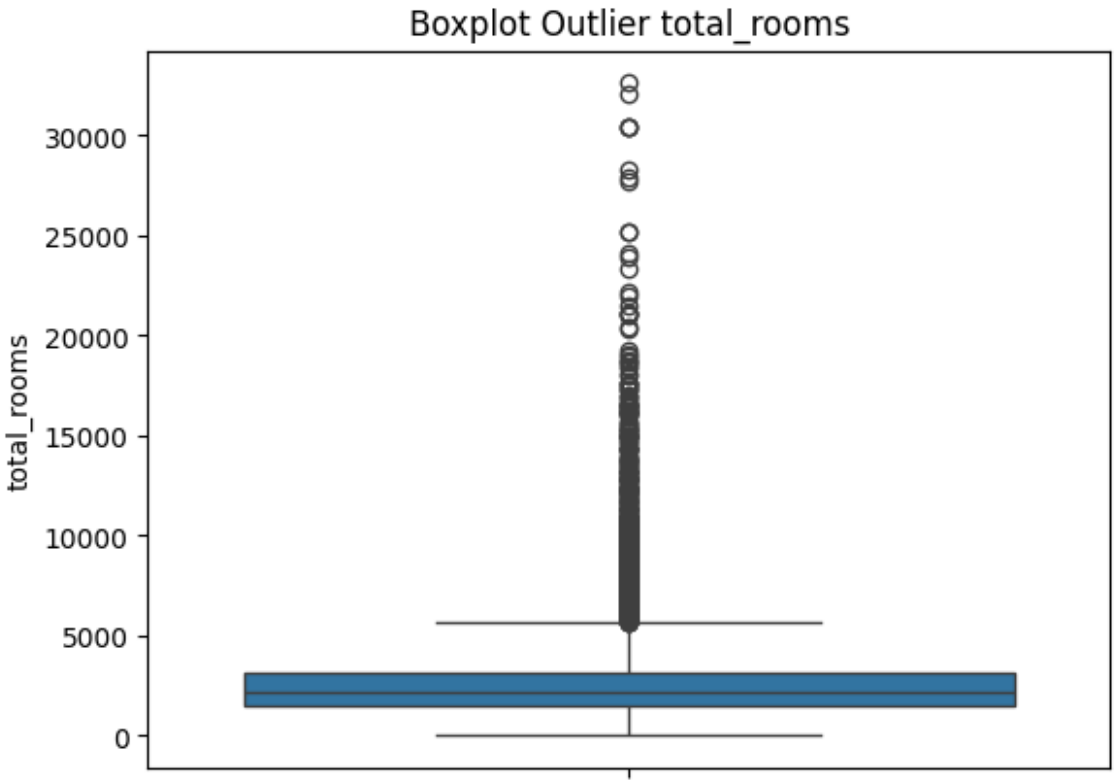
```
outliers_housing_median_age=check_outliers(df,'housing_median_age')
outliers_housing_median_age
```




```
'Tidak Ada Outliers'

## Mengecek perumahan di atas nilai Q3 atau di atas 37 tahun, untuk
mengetahui berapa jarak selisih antar nilainya.
df[df['housing_median_age']>37]['housing_median_age'].value_counts()

housing_median_age
52.0    896
38.0    277
42.0    270
44.0    256
39.0    247
43.0    244
41.0    221
40.0    211
45.0    189
46.0    176
47.0    132
48.0    111
50.0     99
49.0     93
```

```
51.0     36
Name: count, dtype: int64
```

```
## Mengecek outliers pada fitur total_rooms
sns.boxplot(df['total_rooms'])
plt.title('Boxplot Outlier total_rooms')
plt.show()

## Codingan untuk menampilkan data frame yang miliki outlier untuk
kolom total_rooms
outliers_total_rooms=check_outliers(df,'total_rooms')
outliers_total_rooms.head()
```




```
    longitude  latitude  housing_median_age  total_rooms
total_bedrooms  \
5     -117.79     33.80                11.0      10535.0
1620.0
9     -122.27     37.97                10.0      15259.0
2275.0
13    -117.50     33.96                12.0       7923.0
1470.0
22    -121.53     38.61                 5.0       8149.0
1913.0
45    -117.59     33.91                 7.0      10223.0
1491.0
```

```
    population  households  median_income ocean_proximity
median_house_value
5       4409.0      1622.0         6.6700       <1H OCEAN
283200.0
9       7266.0      2338.0         6.0666        NEAR BAY
272400.0
13      4861.0      1385.0         4.2985          INLAND
139200.0
22      2933.0      1616.0         3.6788          INLAND
178800.0
45      5205.0      1509.0         5.4872          INLAND
203400.0
```

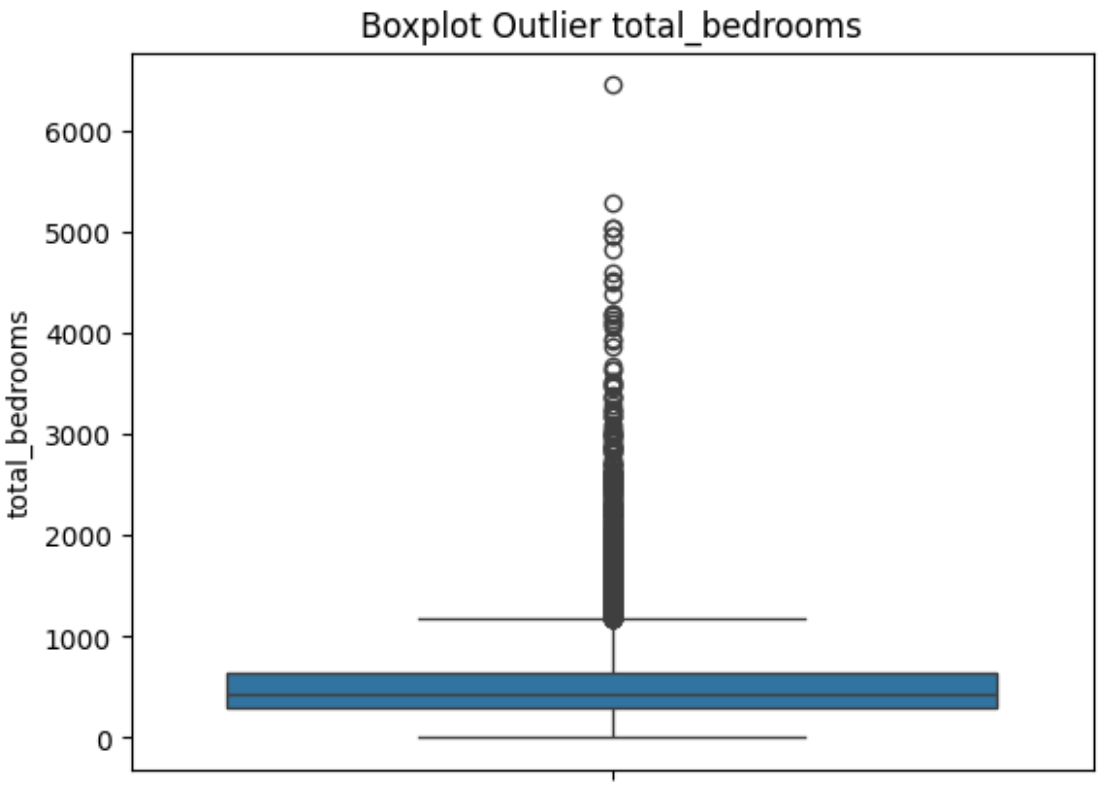
```
## Mencetak informasi total outlier dari kolom total_rooms
print(f'Total outlier pada kolom total_rooms :
{outliers_total_rooms.shape[0]}')
print(f'Total data awal: {df.shape[0]}')
print(f'Persentase total outlier dari total data (%):
{(outliers_total_rooms.shape[0]/df.shape[0])*100}')
```

```
Total outlier pada kolom total_rooms : 900
Total data awal: 14448
Persentase total outlier dari total data (%): 6.229235880398671
```

Dari informasi kolom **total_rooms** terdapat outlier sebanyak 6,3 % atau setara 849 data namun outlier ini tidak dihapus karena setiap rumah dalam satu blok perumahan wajar memiliki total ruangan yang banyak dan jumlahnya berbeda beda. Selain itu nilai sebuah rumah secara kasar diperkirakan dalam hitungan meter persegi.Katakanlah rumah seluas 2.000 meter persegi dijual seharga $ 200.000. Harga per meter persegi adalah $100.

Selain luas persegi, ruang rumah yang dapat digunakan penting saat menentukan nilainya. Garasi, loteng, dan ruang bawah tanah yang belum selesai umumnya tidak dihitung dalam ukuran persegi yang dapat digunakan. Jadi, jika Anda memiliki rumah seluas 2.000 meter persegi dengan garasi seluas 600 kaki persegi, itu hanya ruang yang layak huni seluas 1.400 kaki persegi.

```
## Mengecek outliers pada fitur total_bedrooms
sns.boxplot(df['total_bedrooms'])
plt.title('Boxplot Outlier total_bedrooms')
plt.show()

## Codingan untuk menampilkan data frame yang miliki outlier untuk
kolom total_bedrooms
outliers_total_bedrooms=check_outliers(df,'total_bedrooms')
outliers_total_bedrooms.head()
```



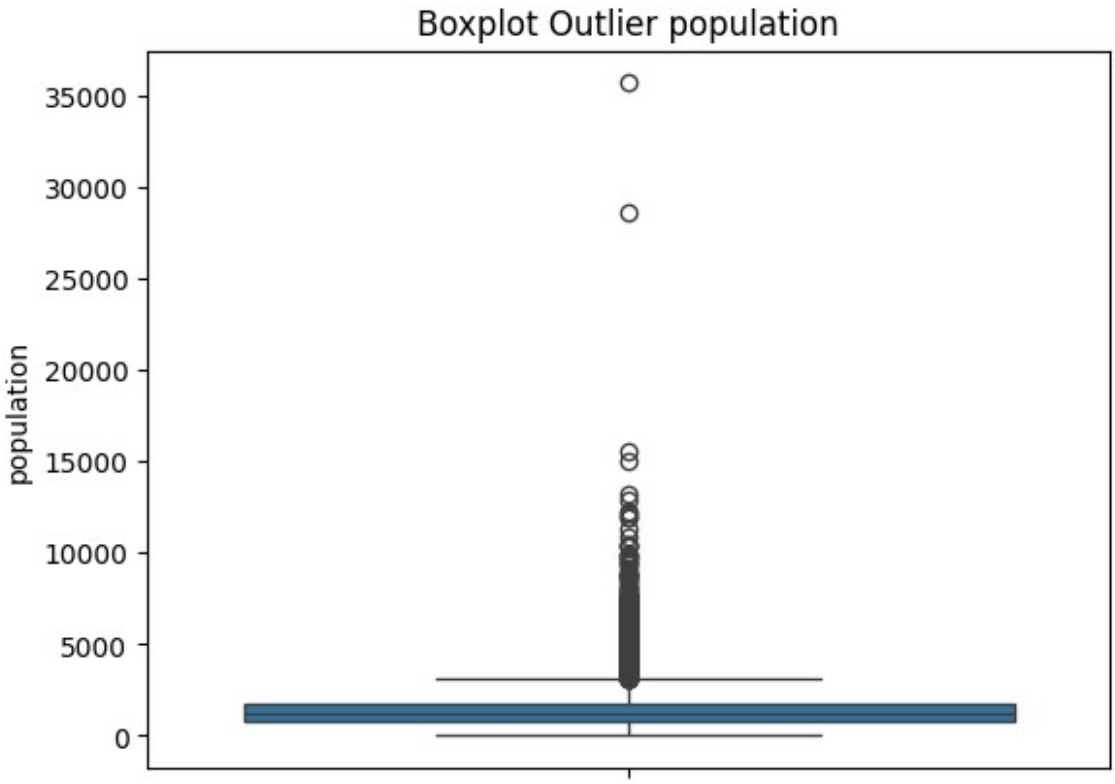

```
    longitude  latitude  housing_median_age  total_rooms
total_bedrooms  \
3     -118.28     34.06                17.0       2518.0
1196.0
5     -117.79     33.80                11.0      10535.0
1620.0
9     -122.27     37.97                10.0      15259.0
2275.0
10    -121.94     37.28                18.0       4356.0
1334.0
13    -117.50     33.96                12.0       7923.0
1470.0

    population  households  median_income ocean_proximity
median_house_value
3       3051.0      1000.0         1.7199       <1H OCEAN
175000.0
5       4409.0      1622.0         6.6700       <1H OCEAN
283200.0
9       7266.0      2338.0         6.0666        NEAR BAY
272400.0
10      1968.0      1245.0         3.6294       <1H OCEAN
240000.0
13      4861.0      1385.0         4.2985          INLAND
139200.0
```

```
## Mencetak informasi total outlier dari kolom total_bedrooms
print(f'Total outlier pada kolom total_bedrooms :
{outliers_total_bedrooms.shape[0]}')
print(f'Total data awal: {df.shape[0]}')
print(f'Persentase total outlier dari total data (%):
{(outliers_total_bedrooms.shape[0]/df.shape[0])*100}')
```

```
Total outlier pada kolom total_bedrooms : 879
Total data awal: 14448
Persentase total outlier dari total data (%): 6.083887043189369
```

Dari informasi kolom **total_bedrooms**, terdapat outlier sebanyak 820 data atau setara 6,1 % outliernya, namun outlier ini tidak dihapus karena pertimbangan jumlah kamar tidur setiap rumah berbeda-beda dan hal ini dipengaruhi dari jumlah orang yang tinggal dirumah tersebut.

```
## Mengecek outliers pada fitur population
sns.boxplot(df['population'])
plt.title('Boxplot Outlier population')
plt.show()

## Codingan untuk menampilkan data frame yang miliki outlier untuk
kolom population
outliers_population=check_outliers(df,'population')
outliers_population.head()
```



```
    longitude  latitude  housing_median_age  total_rooms
total_bedrooms  \
5     -117.79     33.80                11.0      10535.0
1620.0
9     -122.27     37.97                10.0      15259.0
2275.0
13    -117.50     33.96                12.0       7923.0
1470.0
45    -117.59     33.91                 7.0      10223.0
1491.0
54    -121.36     38.67                 5.0       5819.0
1507.0

    population  households  median_income ocean_proximity
median_house_value
5       4409.0      1622.0         6.6700       <1H OCEAN
283200.0
9       7266.0      2338.0         6.0666        NEAR BAY
272400.0
13      4861.0      1385.0         4.2985          INLAND
139200.0
45      5205.0      1509.0         5.4872          INLAND
203400.0
54      3237.0      1356.0         2.2339          INLAND
116600.0
```

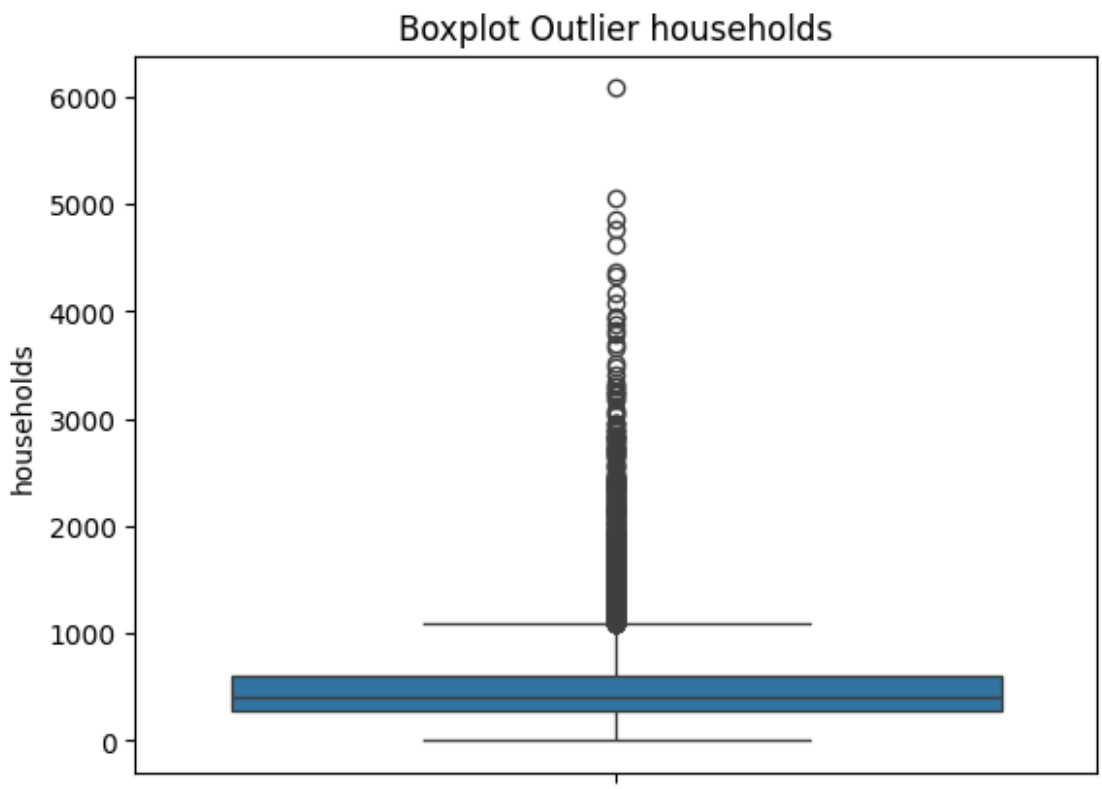
```
## Mencetak informasi total outlier dari kolom population
print(f'Total outlier pada kolom population :
{outliers_population.shape[0]}')
print(f'Total data awal: {df.shape[0]}')
print(f'Persentase total outlier dari total data (%):
{(outliers_population.shape[0]/df.shape[0])*100}')
```

```
Total outlier pada kolom population : 827
Total data awal: 14448
Persentase total outlier dari total data (%): 5.723975636766335
```

Dari informasi kolom **population**, terdapat outlier sebanyak 759 data atau setara 5,6 % outliernya, namun outlier ini tidak dihapus karena pertimbangan jumlah orang yang tinggal disuatu blok perumahan berbeda beda. Selain itu pertimbangan lain ada pada lingkungan perumahan, jika lingkungan perumahan elit maka populasi orang tinggal disana sedikit dan jika lingkungan perumahan tidak elit umumnya populasi orang yang tinggal disana banyak.

```
## Mengecek outliers pada fitur households
sns.boxplot(df['households'])
plt.title('Boxplot Outlier households')
plt.show()

## Codingan untuk menampilkan data frame yang miliki outlier untuk
```

```
kolom households
outliers_households=check_outliers(df,'households')
outliers_households.head()
```




```
    longitude  latitude  housing_median_age  total_rooms
total_bedrooms  \
5     -117.79     33.80                11.0      10535.0
1620.0
9     -122.27     37.97                10.0      15259.0
2275.0
10    -121.94     37.28                18.0       4356.0
1334.0
13    -117.50     33.96                12.0       7923.0
1470.0
22    -121.53     38.61                 5.0       8149.0
1913.0

    population  households  median_income ocean_proximity
median_house_value
5       4409.0      1622.0         6.6700       <1H OCEAN
283200.0
9       7266.0      2338.0         6.0666        NEAR BAY
272400.0
10      1968.0      1245.0         3.6294       <1H OCEAN
```

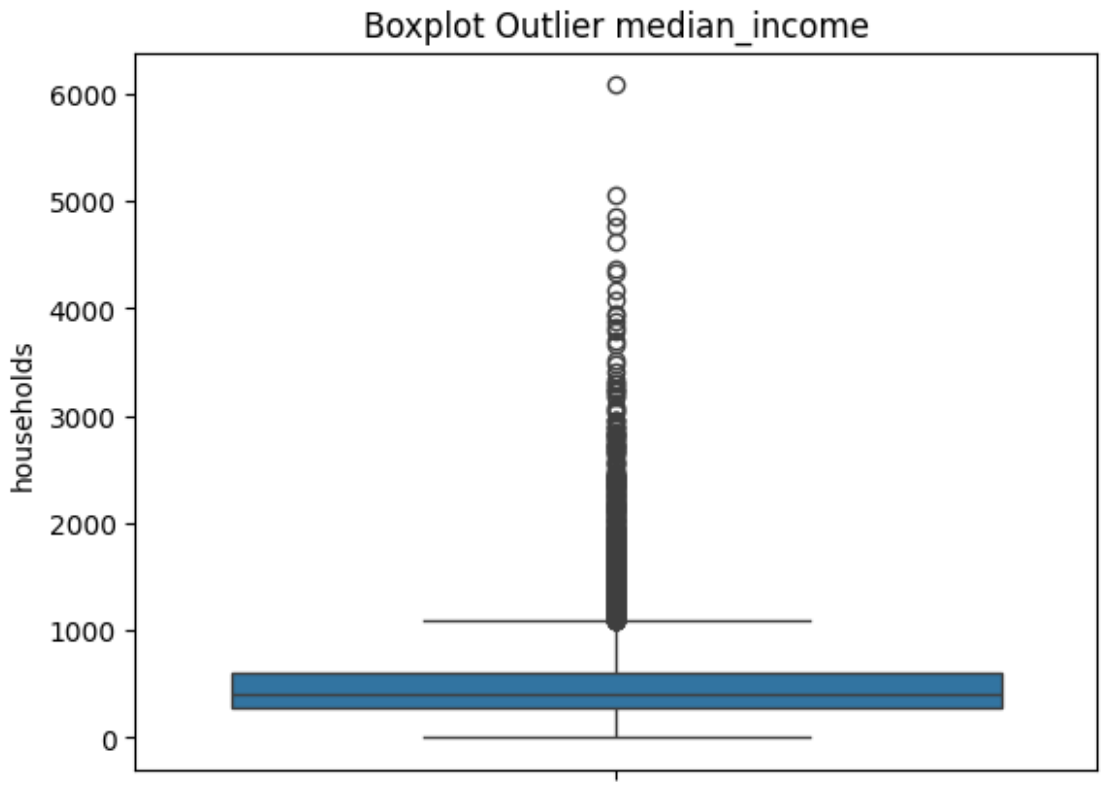
```
240000.0
13      4861.0      1385.0         4.2985          INLAND
139200.0
22      2933.0      1616.0         3.6788          INLAND
178800.0
```

```python
## Mencetak informasi total outlier dari kolom households
print(f'Total outlier pada kolom households :
{outliers_households.shape[0]}')
print(f'Total data awal: {df.shape[0]}')
print(f'Persentase total outlier dari total data (%):
{(outliers_households.shape[0]/df.shape[0])*100}')
```

```
Total outlier pada kolom households : 842
Total data awal: 14448
Persentase total outlier dari total data (%): 5.827796234772979
```

Dari informasi kolom **households**, terdapat outlier sebanyak 799 data atau setara 5,9 % dibulatkan menjadi sekitar 6% outliernya, namun outlier ini tidak dihapus. Karena untuk kepala keluarga dalam 1 blok perumahan berbeda beda dan tidak bisa di batasi, jika daerah perumahan yang tidak elit kemungkinan besar jumlah kepala keluarga nya banyak.

```python
## Mengecek outliers pada fitur median_income
sns.boxplot(df['households'])
plt.title('Boxplot Outlier median_income')
plt.show()

## Codingan untuk menampilkan data frame yang miliki outlier untuk
kolom median_income
outliers_median_income=check_outliers(df,'median_income')
outliers_median_income.head()
```



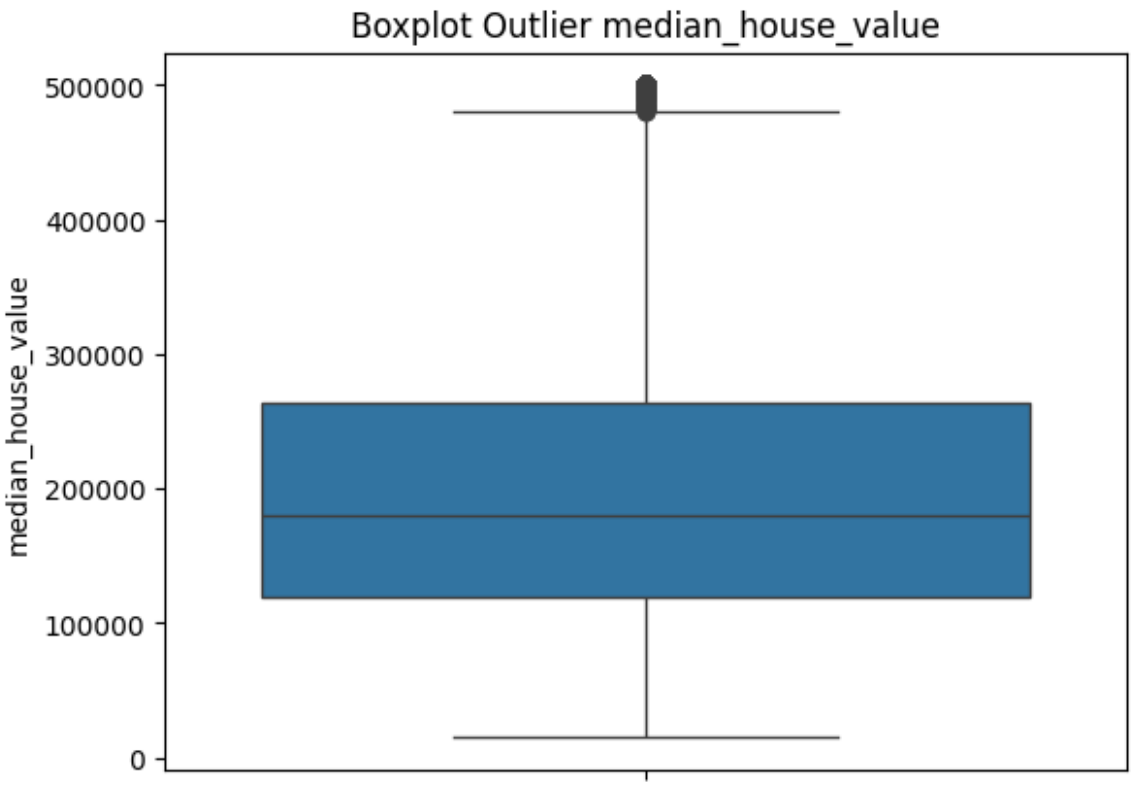

```
     longitude  latitude  housing_median_age  total_rooms
total_bedrooms  \
6      -117.80     33.81                14.0       1206.0
142.0
8      -117.65     33.48                 6.0       1638.0
188.0
43     -122.21     37.37                34.0       1476.0
217.0
73     -121.99     37.26                17.0       4034.0
611.0
124    -121.89     37.23                20.0       7754.0
976.0

     population  households  median_income ocean_proximity
median_house_value
6         572.0       149.0         8.8470       <1H OCEAN
388700.0
8         572.0       174.0        13.0502       <1H OCEAN
500001.0
43        613.0       223.0         8.2883      NEAR OCEAN
500001.0
73       1158.0       560.0         8.2069       <1H OCEAN
442500.0
124      3094.0       941.0         8.1900       <1H OCEAN
361600.0
```

```
## Mencetak informasi total outlier dari kolom median_income
print(f'Total outlier pada kolom median_income :
{outliers_median_income.shape[0]}')
print(f'Total data awal: {df.shape[0]}')
print(f'Persentase total outlier dari total data (%):
{(outliers_median_income.shape[0]/df.shape[0])*100}')
```

```
Total outlier pada kolom median_income : 482
Total data awal: 14448
Persentase total outlier dari total data (%): 3.3361018826135105
```

Dari informasi kolom **median_income**, terdapat outlier sebanyak 421 data atau setara 3,1 %, outlier ini tidak dihapus karena ini informasi terkait pendapatan orang rata rata dalam 1 blok dapat memberikan gambaran tentang blok perumahan tersebut masuk dalam perumahan mewah atau tidak dan umumnya orang yang memiliki pendapatan tinggi akan berada pada kawasan mewah.

```
## Mengecek outliers pada fitur median_house_value
sns.boxplot(df['median_house_value'])
plt.title('Boxplot Outlier median_house_value')
plt.show()

## Codingan untuk menampilkan data frame yang miliki outlier untuk
kolom median_house_value
outliers_median_house_value=check_outliers(df,'median_house_value')
outliers_median_house_value.head()
```




```
     longitude  latitude  housing_median_age  total_rooms
total_bedrooms  \
8      -117.65     33.48                 6.0       1638.0
188.0
43     -122.21     37.37                34.0       1476.0
217.0
106    -117.92     33.63                39.0       1469.0
226.0
133    -122.44     37.80                52.0       3257.0
735.0
158    -118.38     34.14                40.0       1965.0
354.0

     population  households  median_income ocean_proximity
median_house_value
8         572.0       174.0        13.0502       <1H OCEAN
500001.0
43        613.0       223.0         8.2883      NEAR OCEAN
500001.0
106       553.0       225.0         7.8496       <1H OCEAN
490800.0
133      1045.0       620.0         4.5523        NEAR BAY
500001.0
158       666.0       357.0         6.0876       <1H OCEAN
483800.0
```

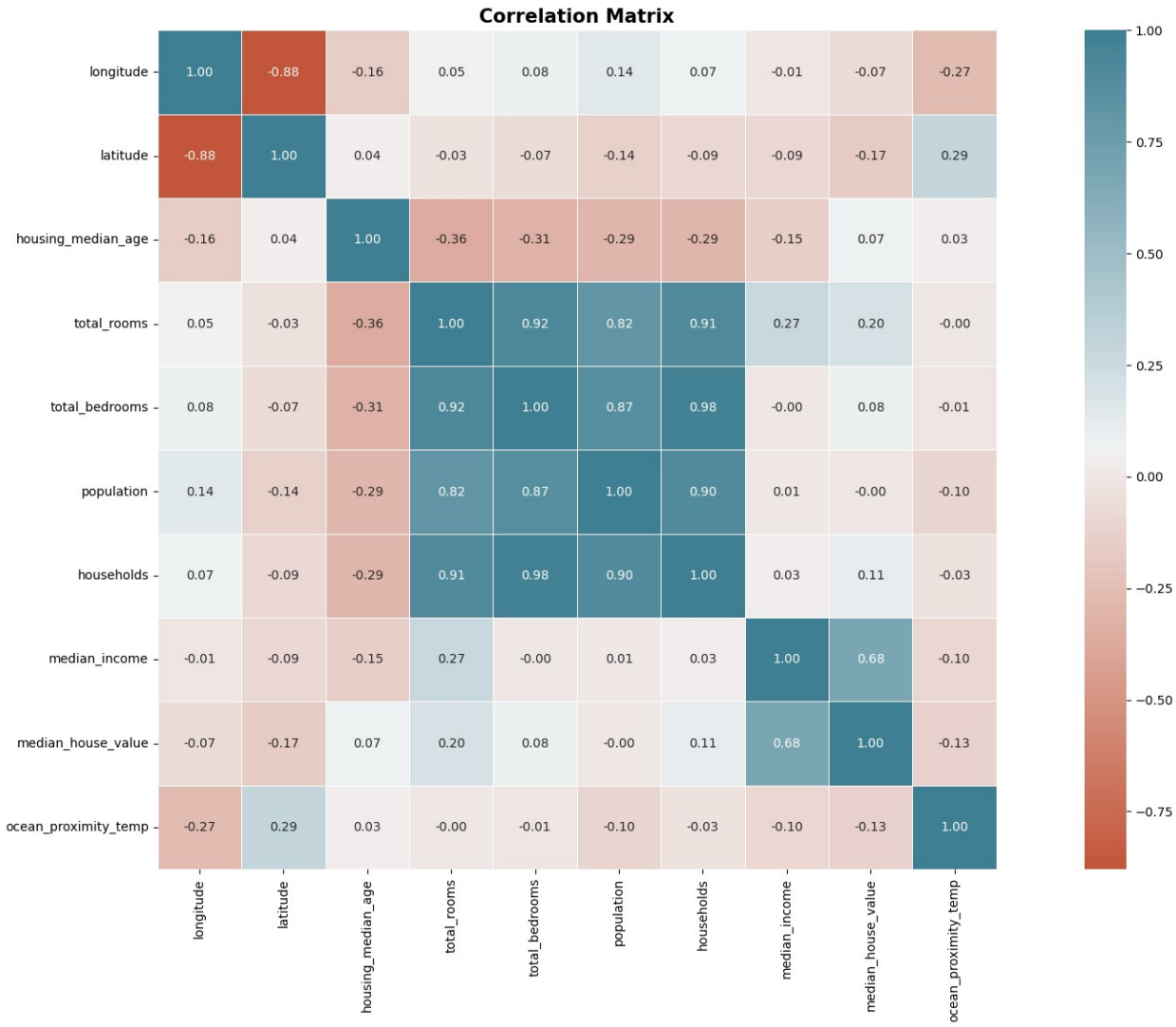
```python
## Mencetak informasi total outlier dari kolom median_house_value
print(f'Total outlier pada kolom median_house_value :
{outliers_median_house_value.shape[0]}')
print(f'Total data awal: {df.shape[0]}')
print(f'Persentase total outlier dari total data (%):
{(outliers_median_house_value.shape[0]/df.shape[0])*100}')
```

```
Total outlier pada kolom median_house_value : 757
Total data awal: 14448
Persentase total outlier dari total data (%): 5.239479512735326
```

```python
## Mengecek harga maksimal rumah setelah hapus outlier
df['median_house_value'].describe()
```

```
count     14448.000000
mean     206824.624516
std      115365.476182
min       14999.000000
25%      119600.000000
50%      180000.000000
75%      263900.000000
max      500001.000000
Name: median_house_value, dtype: float64
```

```python
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df['ocean_proximity_temp'] = le.fit_transform(df['ocean_proximity'])

plt.figure(figsize=(25,12))

palette = sns.diverging_palette(20,220,n=256)

corr = df.select_dtypes(include='number').corr(method='spearman')

sns.heatmap(
    corr,
    annot=True,
    fmt='.2f',
    cmap=palette,
    square=True,
    linewidths=0.5
)

plt.title('Correlation Matrix', size=15, weight='bold')
plt.show()
```



- Jika dilihat dari data di atas :
  - longtitude dan latitude memiliki memiliki **korelasi negatif dengan kategori lemah,** jadi dimana letak lokasi perumahan tidak mempengaruhi kenaikan harga rumah.
  - house_median_age, total_rooms, total_bedrooms, population, households memiliki **korelasi positif namun masuk dalam kategori lemah**, artinya semakin tinggi nilai dari median_house_value maka semakin tinggi juga nilai house_median_age, total_rooms, total_bedrooms, population, households.
  - median_income memiliki **korelasi positif dan masuk dalam kategori sedang**, artinya semakin tinggi nilai dari median_house_value maka semakin tinggi juga nilai median_income.

2.3 Menentukan Objek Data

- **Atribut Prediktor (Fitur):**
  Fitur seperti `longitude`, `latitude`, `housing_median_age`, `total_rooms`, `total_bedrooms`, `population`, `households`, `median_income`, dan

`ocean_proximity` akan digunakan sebagai fitur utama untuk melatih model prediksi harga rumah.

- **Atribut Target:**
  Kolom `median_house_value` menjadi fokus prediksi (Target Kontinu) untuk memperkirakan nilai pasar rumah di masing-masing blok.

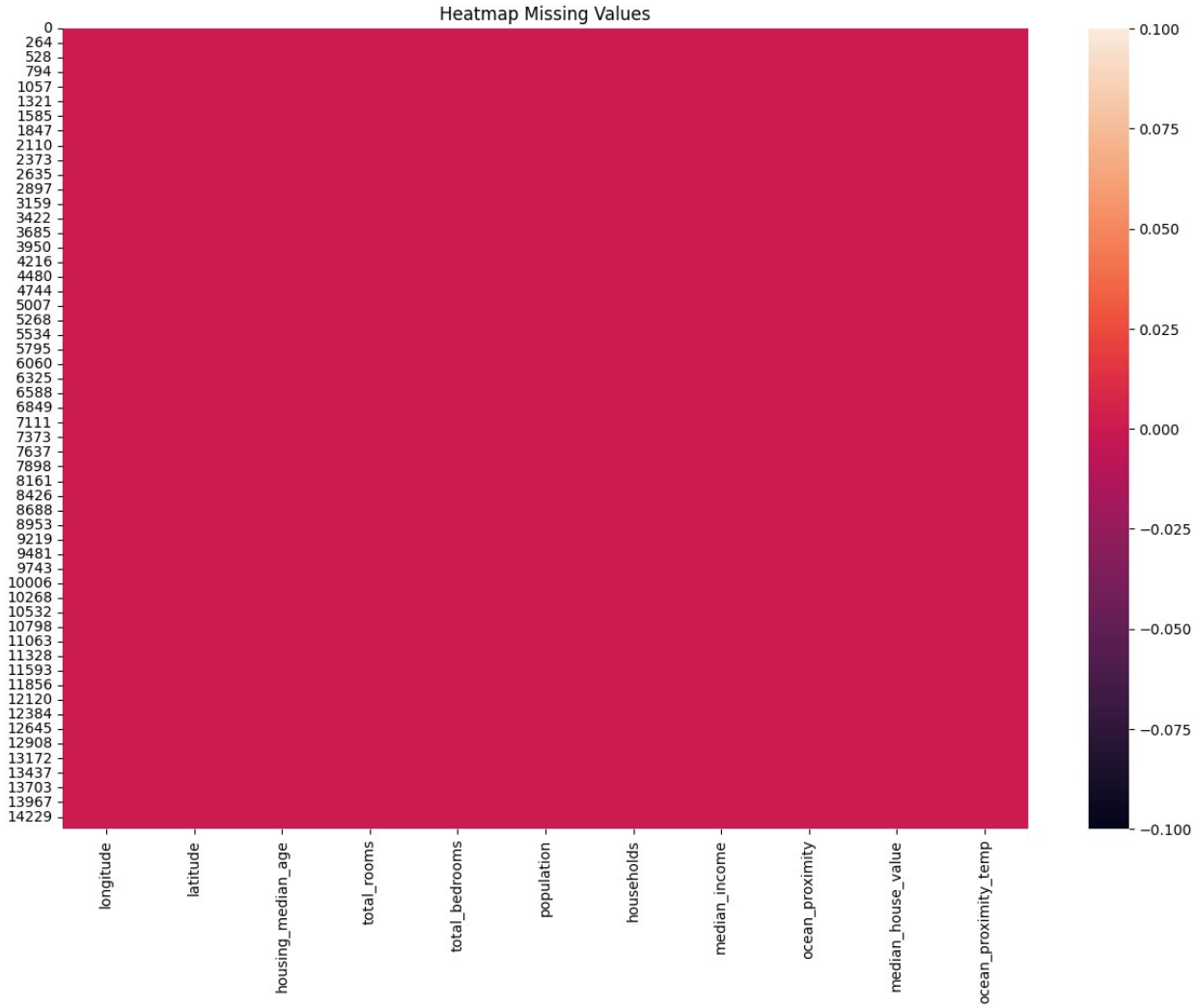
2.4 Membersihkan Data

- Melakukan pembersihan data yang kotor
- Membuat laporan dan rekomendasi hasil membersihkan data

```
total_baris = round(df['total_bedrooms'].notnull().sum() * 100 /
len(df),3)
total_baris_missing_values = round(df['total_bedrooms'].isnull().sum()
* 100 / len(df['total_bedrooms']),3)
print(f'Total persentase data awal kolom total_bedrooms (termasuk
missing values) \t : {total_baris_missing_values+total_baris} % ')
print(f'Total persentase missing values kolom total_bedrooms \t\t\t\
t : {total_baris_missing_values} %')
print(f'Total persentase kolom total_bedrooms yang tidak memiliki
missing values \t : {total_baris} %')
```

```
Total persentase data awal kolom total_bedrooms (termasuk missing
values)     : 100.0 %
Total persentase missing values kolom total_bedrooms
      : 0.948 %
Total persentase kolom total_bedrooms yang tidak memiliki missing
values      : 99.052 %
```

```
df.dropna(inplace=True)

plt.figure(figsize=(15,10))
sns.heatmap(df.isna(),cbar='YlOrRd_r')
plt.title('Heatmap Missing Values')
plt.show()
```




```python
# Mengubah tipe data kolom total_rooms
df['total_rooms'] = df['total_rooms'].astype('int64')

# Mengubah tipe data kolom total_bedrooms
df['total_bedrooms'] = df['total_bedrooms'].astype('int64')

# Mengubah tipe data kolom population
df['population'] = df['population'].astype('int64')

# Mengubah tipe data kolom households
df['households'] = df['households'].astype('int64')
```

- Tipe data yang akan di ubah adalah :
  - total_rooms
  - total_bedrooms

  - population
  - households

- **Alasan mengganti tipe** data dari **float** ke **int** karena total_rooms,total_bedrooms,population,households berdasarkan domain knowladge masuk dalam jenis angka dikret atau angka satuan yang tidak memiliki pecahan decimal.

  - Contoh total_rooms : jumlah total_rooms ini kurang tepat jika di katakan total ada ruangan 1,2 ruangan. Dalam menghitung jumlah ruangan kurang tepat jika menggunakan pecahan (jenis angka continous) dan kurang logis sehingga lebih tepat menggunakan tipe data int agar datanya menjadi jenis diskret (satuan yang tidak memiliki pecahan decimal).

  - Contoh total_bedrooms : jumlah total_bedrooms juga kurang tepat jika menggunakan tipe data float karena jenis angkanya akan menjadi jenis angka continous, akan lebih tepat jika menggunakan tipe data int agar jenis angkanya menjadi jenis angka diskret (angka satuan tanpa decimal). Karena kurang tepat jika total ruangan dikatakan 1,5 jumlah ruangan karena ini menghitung jumlah satuan bukan jumlah meter. Sehingga akan lebih tepat ialah jumlah ruangan menggunakan jenis angka diskret misalnya jumlah ruangan ada 1 atau jumlah ruangan ada 5 (tidak menggunakan pecahan decimal).

  - Contoh populasi : jumlah populasi juga kurang tepat jika menggunakan tipe data float karena ini angkanya akan berubah ke continous, akan lebih tepat jika menggunakan tipe data int agar angkanya berubah menjadi satuan atau jenis angka diskret. Karena pada populasi itu menggambarkan jumlah manusia, manusia itu tidak dapat dibagi menjadi setengah karena kalau menghitung manusia itu masuknya jenis angka satuan yang tidak memiliki pecahan decimal.

- Contoh households : jumlah households ini merupakan jumlah orang dalam rumah yang berada dalam 1 blok, sehingga kurang tepat jika menggunakan tipe data float dan akan lebih tepat jika menggunakan tipe data int agar angkanya menjadi angka diskret. Jenis angka diskret lebih tepat digunakan untuk menghitung jumlah manusia karena manusia dihitung dalam satuan yang tidak memiliki pecahan decimal.

Jika di lihat dari visualisasi di atas sudah tidak ada missing values, artinya missing values sudah di hapus semua.

```
# Menghapus outlier housing_median_age
df = df[(df['housing_median_age'] < 51)]
```

Jika dilihat dari umur perumahan diatas, outlier terbanyak ada pada umur rumah 52 tahun karena selisih antar umur 52 dan 51 tahun dengan umur perumahan dibawahnya mengalami selisih yang sangat jauh. Disini saya membatasi umur rumah maximal yang dapat digunakan dalam model adalah 50 tahun. Selain karena outlier, yang menjadi pertimbangan berikutnya adalah rumah yang diatas 50 tahun berdiri sangat tua sudah dan perlu dilakukan renovasi rumah seperti atap rumah hanya bertahan 20-30 tahun seperti yang dikutip dari *web onethehouse*, selain itu umur perumahan rata-rata dari segi bagunannya hanya bertahan sampai dengan 50-60, jika sudah masuk dalam umur ini maka sudah saatnya dilakukan pengecekan untuk renovasi seperti yang dikutip dari *website makaaniq*.

```
## Mengecek harga maksimal rumah sebelum hapus outlier
df['median_house_value'].describe()
```

```
count     13384.000000
mean     202110.104603
std      111980.553989
min       14999.000000
25%      118200.000000
50%      176700.000000
75%      255925.000000
max      500001.000000
Name: median_house_value, dtype: float64
```

```
df.head()
```

```
   longitude  latitude  housing_median_age  total_rooms
total_bedrooms  \
1    -122.21     37.77                43.0         1017
328
2    -118.04     33.87                17.0         2358
396
3    -118.28     34.06                17.0         2518
1196
4    -119.81     36.73                50.0          772
```

```
194
5    -117.79     33.80                11.0        10535
1620

   population  households  median_income ocean_proximity
median_house_value  \
1         836         277         2.2604        NEAR BAY
100000.0
2        1387         364         6.2990       <1H OCEAN
285800.0
3        3051        1000         1.7199       <1H OCEAN
175000.0
4         606         167         2.2206          INLAND
59200.0
5        4409        1622         6.6700       <1H OCEAN
283200.0

   ocean_proximity_temp
1                     3
2                     0
3                     0
4                     1
5                     0
```

```
## Drop outliers data of median_house_value
df.drop(outliers_median_house_value.index, inplace=True,
errors='ignore')

df.describe().sort_values(by='median_house_value', ascending=False)
```

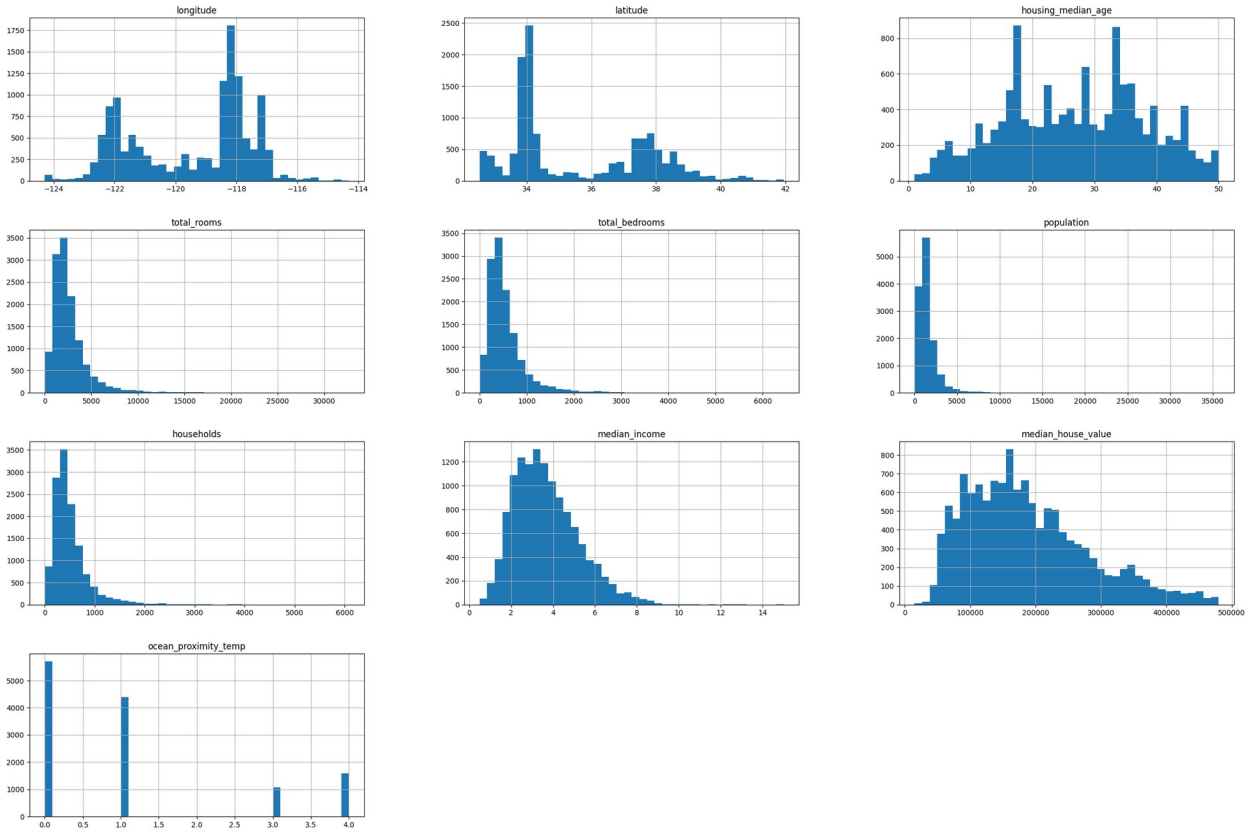
```
          longitude      latitude  housing_median_age   total_rooms  \
max     -114.310000     41.950000           50.000000  32627.000000
75%     -117.970000     37.670000           36.000000   3184.000000
mean    -119.474489     35.588768           26.868827   2675.591062
50%     -118.450000     34.240000           27.000000   2143.000000
25%     -121.520000     33.920000           17.000000   1459.000000
std        1.979656      2.150569           11.383892   2239.036625
min     -124.300000     32.540000            1.000000      2.000000
count  12777.000000  12777.000000        12777.000000  12777.000000

       total_bedrooms    population    households  median_income  \
max       6445.000000  35682.000000   6082.000000      15.000100
75%        661.000000   1779.000000    615.000000       4.591800
mean       549.463802   1472.514675    509.524928       3.678131
50%        443.000000   1206.000000    416.000000       3.469500
25%        299.000000    810.000000    283.000000       2.539500
std        433.521681   1184.932322    392.598518       1.545646
min          2.000000      3.000000      2.000000       0.499900
count    12777.000000  12777.000000  12777.000000   12777.000000
```

```
       median_house_value  ocean_proximity_temp
max          480100.000000              4.000000
75%          241400.000000              1.000000
mean         188003.333881              1.096658
50%          170900.000000              1.000000
25%          115300.000000              0.000000
std           93523.490806              1.378042
min           14999.000000              0.000000
count         12777.000000          12777.000000
```

Dari informasi kolom **median_house_value**, terdapat outlier sebanyak 663 data atau setara 4,9 %, outlier ini dihapus karena ini adalah target yang harusnya bias pada target sedikit sehingga alasan ini saya memutuskan untuk menghapus outlier. Awalnya harga rumah tertinggi 500001 US Dollar namun saya hapus outlier dan harga rumah tertinggi sekarang adalah 480100 US Dollar.

```
## Visualisasi data untuk mengecek distribusi data
df.hist(bins=40,figsize=(30,20))
```

```
array([[<Axes: title={'center': 'longitude'}>,
        <Axes: title={'center': 'latitude'}>,
        <Axes: title={'center': 'housing_median_age'}>],
       [<Axes: title={'center': 'total_rooms'}>,
        <Axes: title={'center': 'total_bedrooms'}>,
        <Axes: title={'center': 'population'}>],
       [<Axes: title={'center': 'households'}>,
        <Axes: title={'center': 'median_income'}>,
        <Axes: title={'center': 'median_house_value'}>],
       [<Axes: title={'center': 'ocean_proximity_temp'}>, <Axes: >,
        <Axes: >]], dtype=object)
```



Berdasarkan dari visualisasi data di atas adapat dilihat data tidak berdistribusi normal.

Atribut	Masalah	Strategi	Justifikasi Teknis
total_bedrooms	Terdapat missing values sebanyak 137 baris (0.948%) dari 14.448 total baris	Menghapus baris dengan missing values	Jumlah missing values sangat kecil (<1%) sehingga penghapusan tidak signifikan memengaruhi model. Sisa data 14.331 baris (99,052%) tetap representatif untuk pelatihan model machine learning.
total_rooms	Tipe data float, tidak sesuai dengan konteks	Mengubah tipe data dari float ke int	Jumlah ruangan dihitung sebagai satuan, tidak logis jika menggunakan pecahan. Tipe int lebih tepat untuk angka diskret dan menghitung total ruangan.
total_bedrooms	Tipe data float	Mengubah tipe data dari float ke int	Jumlah kamar tidur adalah angka diskret; float akan mengubahnya menjadi angka kontinu yang tidak sesuai dengan domain knowledge. Tipe int merepresentasikan jumlah kamar secara nyata (misal 1, 2, 3 kamar).
population	Tipe data float	Mengubah tipe data dari	Populasi merepresentasikan jumlah manusia; manusia tidak dapat dibagi setengah. Tipe int menjadikan data diskret dan logis untuk

Atribut	Masalah	Strategi	Justifikasi Teknis
		float ke int	perhitungan jumlah orang.
households	Tipe data float	Mengubah tipe data dari float ke int	Households dihitung sebagai jumlah orang per rumah; float akan membuat data menjadi angka kontinu yang tidak relevan. Tipe int menjaga akurasi jumlah unit manusia sebagai angka diskret.

2.5 Mengkonstruksi Data

- Menganalisis teknik transformasi data
- Melakukan transformasi data
- Membuat dokumentasi konstruksi data

```
# Encoding Kategori
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer

# Kolom kategorikal dan numerikal
categorical_column = ['ocean_proximity']
numerical_column = [
    'housing_median_age',
    'total_rooms',
    'total_bedrooms',
    'population',
    'households',
    'median_income'
]

# Pipeline untuk kolom kategorikal
categorical_tranformer = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),  # Isi NaN
dengan modus
    ('onehot', OneHotEncoder(drop='first', handle_unknown='ignore'))
])

# Pipeline untuk kolom numerikal
numerical_transformer = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler(with_mean=False))
])

# Gabungkan pipeline dengan ColumnTransformer
transformer = ColumnTransformer([
    ('cat', categorical_tranformer, categorical_column),
```

```
    ('num', numerical_transformer, numerical_column)
], remainder='passthrough')
```

Preprocessing Data: Pipeline Kategorikal dan Numerikal

1. Kolom Kategorikal dan Numerikal

- **Kolom Kategorikal**: `ocean_proximity`
  → Berisi informasi kategori tentang lokasi rumah relatif terhadap laut.
- **Kolom Numerikal**: `housing_median_age`, `total_rooms`, `total_bedrooms`, `population`, `households`, `median_income`
  → Berisi informasi numerik seperti jumlah kamar, usia rumah, jumlah penduduk, dan pendapatan.

2. Pipeline Kolom Kategorikal

- **Imputasi**:
  Mengisi nilai yang hilang (NaN) dengan modus (kategori yang paling sering muncul).
  Hal ini dilakukan karena **OneHotEncoder tidak bisa memproses nilai kosong**.
- **Encoding**:
  Mengubah kategori menjadi angka menggunakan **OneHotEncoding**.
  - `drop='first'` digunakan untuk menghindari dummy variable trap, yaitu kolinearitas antara kolom dummy.
  - `handle_unknown='ignore'` digunakan agar pipeline tidak error jika muncul kategori baru saat prediksi.

3. Pipeline Kolom Numerikal

- **Imputasi**:
  Mengisi nilai hilang (NaN) dengan **median**.
  Median dipilih karena distribusi kolom numerikal biasanya **tidak normal atau skewed**, sehingga median lebih representatif daripada mean.
- **Scaling**:
  Menormalkan nilai numerik menggunakan **StandardScaler** agar skala setiap kolom setara.
  Hal ini membantu model machine learning untuk **tidak bias terhadap fitur dengan skala besar**.
  - `with_mean=False` dipakai agar pipeline aman jika nanti dikombinasikan dengan sparse matrix hasil encoding.

4. Menggabungkan Pipeline

- Pipeline kategorikal dan numerikal digabungkan menggunakan **ColumnTransformer**.
- Setiap pipeline diterapkan hanya pada kolom yang sesuai, sementara kolom lain tetap dilewatkan (`remainder='passthrough'`).
- Hasilnya adalah **data siap pakai untuk model machine learning**, di mana semua fitur numerik telah diskalakan dan kategori telah diubah menjadi angka.

Ringkasan

1. Data kategori → diisi NaN dengan modus → diubah menjadi dummy variables

2. Data numerik → diisi NaN dengan median → diskalakan agar setara

3. Semua pipeline digabung → memudahkan preprocessing sebelum training model

2.6 Membangun Skenario Model

- Mengidentifikasi teknik pemodelan
- Menentukan teknik pemodelan yang sesuai dengan karakteristik data dan tujuan teknis data science
- Menyiapkan skenario pengujian

Dalam membangun skenario model prediksi harga rumah, langkah-langkah yang dilakukan meliputi:

- **Identifikasi Teknik Pemodelan:**
  Beberapa algoritma regresi yang dipertimbangkan untuk prediksi `median_house_value` meliputi:
  - `LinearRegression`
  - `KNeighborsRegressor`
  - `DecisionTreeRegressor`
  - `RandomForestRegressor`
  - `XGBRegressor`
- **Persiapan Data:**
  - **Encoding Kategori:** Kolom `ocean_proximity` berisi informasi kategorikal jarak rumah dengan pantai, diubah menjadi data numerikal menggunakan teknik **One-Hot Encoding**.

  - **Scaling Numerikal:** Semua kolom numerikal dinormalisasi menggunakan **Standard Scaler** agar setiap fitur memiliki skala yang seimbang dan tidak ada penyimpangan besar antar variabel.
- **Pipeline:**
  Digunakan pipeline untuk menggabungkan semua tahap preprocessing dan pemodelan sehingga proses dapat dijalankan secara konsisten dan efisien.

Langkah-langkah ini memastikan skenario pemodelan dapat dilakukan secara sistematis dan model yang dibangun dapat memberikan prediksi harga rumah yang akurat.

2.7 Mengevaluasi Hasil Pemodelan

- **Menyiapkan parameter model**
  Sebelum pemodelan, kita mendefinisikan ruang hyperparameter yang akan dituning untuk mendapatkan model terbaik.

- **Menggunakan tools pemodelan**
  Pipeline dan library seperti `scikit-learn` dan `XGBoost` digunakan untuk membangun model.

- **Pengujian Model:**
  Menggunakan **cross-validation** untuk mengevaluasi performa model secara menyeluruh dan menggunakan metrik evaluasi seperti:
  - `mean_absolute_error (MAE)`
  - `mean_absolute_percentage_error (MAPE)`

  Model terbaik akan dipilih berdasarkan hasil cross-validation, kemudian dilakukan **hyperparameter tuning** untuk meningkatkan performa.

BAB 3 : HASIL DAN PEMBAHASAN

3.1 Membangun Model

- Menggunakan model dengan data riil
- Menilai hasil permodelan

```
!pip install xgboost

# Model Machine Learning
from sklearn.linear_model import LinearRegression
from sklearn.neighbors import KNeighborsRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor

# Target Transformation
from sklearn.compose import TransformedTargetRegressor

# Evaluation Metrics
from sklearn.metrics import (
    mean_absolute_error,
    mean_absolute_percentage_error
)

# Model Selection & Validation
from sklearn.model_selection import (
    train_test_split,
    cross_val_score,
    RandomizedSearchCV,
    GridSearchCV,
    KFold,
    cross_validate
)
```

```
Requirement already satisfied: xgboost in D:\Big Data\venv\Lib\site-
packages (3.2.0)
Requirement already satisfied: numpy in D:\Big Data\venv\Lib\site-
packages (from xgboost) (2.3.5)
Requirement already satisfied: scipy in D:\Big Data\venv\Lib\site-
packages (from xgboost) (1.16.3)
```

```
## Memisahkan data independen variabel dengan target
X = df.drop(['median_house_value'], axis=1)
y = df['median_house_value']

## Splitting data training dan test dengan proporsi 70:30
X_train, X_test, y_train, y_test=
train_test_split(X,y,test_size=0.3,random_state=42)

## Menyiapkan algoritma yang digunakan
model_lr = LinearRegression()
model_knn = KNeighborsRegressor()
model_dt = DecisionTreeRegressor(random_state=42)
model_rf = RandomForestRegressor(random_state=42)
model_xgb = XGBRegressor(random_state=42)

## Pemodelan dilakukan dalam skala logaritmik, namun kemudian di-
inverse kembali untuk interpretasi
log_lr = TransformedTargetRegressor(model_lr, func=np.log,
inverse_func=np.exp)
log_knn = TransformedTargetRegressor(model_knn, func=np.log,
inverse_func=np.exp)
log_dt = TransformedTargetRegressor(model_dt, func=np.log,
inverse_func=np.exp)
log_rf = TransformedTargetRegressor(model_rf, func=np.log,
inverse_func=np.exp)
log_xgb = TransformedTargetRegressor(model_xgb, func=np.log,
inverse_func=np.exp)

## Memasukan kandidat algoritma dalam sebuah variabel
models = [log_lr, log_knn, log_dt, log_rf, log_xgb]

## Untuk menyimpan nilai MAE
score_mae = []
nilai_mean_mae = []
nilai_std_mae = []

## Untuk menyimpan nilai MAPE
score_mape = []
nilai_mean_mape = []
nilai_std_mape = []


## Mencari algoritma terbaik dalam tahap train
for i in models:

    ## metode statistik yang dapat digunakan untuk mengevaluasi
kinerja model
    crossval = KFold(n_splits=5, shuffle=True, random_state=42)
#n_split 5 kombinasi, #shuffle untuk mengacak -> dibagi 5 berdasarkan
n_splits
```

```
    estimator = Pipeline([
        ('preprocessing', transformer),
        ('model_machine_learning', i)
    ])

    ## MAE
    model_cv_mae = cross_val_score(
        estimator,
        X_train,
        y_train,
        cv=crossval,
        scoring='neg_mean_absolute_error',
        error_score='raise' ### Jika diatur ke 'raise', nilai error
dimunculkan
        )
        ### neg_mean_absolute_error ini scoring parameter untuk nanti
dapat di gunakan pada metrik mean_absolute_error
        ### Jika diatur ke 'raise', nilai error dimunculkan

    print(model_cv_mae, i)

    score_mae.append(model_cv_mae)
    nilai_mean_mae.append(model_cv_mae.mean())
    nilai_std_mae.append(model_cv_mae.std())


    ## MAPE
    model_cv_mape = cross_val_score(
        estimator,
        X_train,
        y_train,
        cv=crossval,
        scoring='neg_mean_absolute_percentage_error',
        error_score='raise'
        )
     ### neg_mean_absolute_percentage_error ini scoring parameter
untuk nanti dapat di gunakan pada metrik
mean_absolute_percentage_error
     ### Jika diatur ke 'raise', nilai error dimunculkan

    print(model_cv_mape, i)

    score_mape.append(model_cv_mape)
    nilai_mean_mape.append(model_cv_mape.mean())
    nilai_std_mape.append(model_cv_mape.std())
```

```
[-43390.86732168 -41744.3034359  -41479.50113924 -40261.53581762
 -41175.26416678] TransformedTargetRegressor(func=<ufunc 'log'>,
inverse_func=<ufunc 'exp'>,
```

```
                            regressor=LinearRegression())
[-0.24372442 -0.23750038 -0.2445516  -0.23711875 -0.23321021]
TransformedTargetRegressor(func=<ufunc 'log'>, inverse_func=<ufunc
'exp'>,
                            regressor=LinearRegression())
[-36619.57041182 -35552.25541373 -37002.54217854 -35914.16427516
 -35334.94944436] TransformedTargetRegressor(func=<ufunc 'log'>,
inverse_func=<ufunc 'exp'>,
                            regressor=KNeighborsRegressor())
[-0.20616608 -0.19510874 -0.2091795  -0.20264899 -0.19627697]
TransformedTargetRegressor(func=<ufunc 'log'>, inverse_func=<ufunc
'exp'>,
                            regressor=KNeighborsRegressor())
[-41852.76690889 -45223.30911124 -40681.27501397 -41355.31375839
 -42196.98042506] TransformedTargetRegressor(func=<ufunc 'log'>,
inverse_func=<ufunc 'exp'>,

regressor=DecisionTreeRegressor(random_state=42))
[-0.24679423 -0.25488688 -0.24175957 -0.23937399 -0.24594693]
TransformedTargetRegressor(func=<ufunc 'log'>, inverse_func=<ufunc
'exp'>,

regressor=DecisionTreeRegressor(random_state=42))
[-30087.02755811 -30491.1036289  -30751.05715746 -29800.76056386
 -30610.40597835] TransformedTargetRegressor(func=<ufunc 'log'>,
inverse_func=<ufunc 'exp'>,

regressor=RandomForestRegressor(random_state=42))
[-0.17187424 -0.1697166  -0.17637887 -0.17229097 -0.17096115]
TransformedTargetRegressor(func=<ufunc 'log'>, inverse_func=<ufunc
'exp'>,

regressor=RandomForestRegressor(random_state=42))
[-29063.50216165 -29431.52627603 -29749.68622921 -29060.91551096
 -29122.76949629] TransformedTargetRegressor(func=<ufunc 'log'>,
inverse_func=<ufunc 'exp'>,
                            regressor=XGBRegressor(base_score=None,
booster=None,
                                                   callbacks=None,

colsample_bylevel=None,

colsample_bynode=None,

colsample_bytree=None,
                                                   device=None,

early_stopping_rounds=None,

enable_categorical=False,
```

```
                                                        eval_metric=None,
                                                        feature_types=None,

feature_weights=None,
                                                        gamma=None,
grow_policy=None,

importance_type=None,

interaction_constraints=None,
                                                        learning_rate=None,
                                                        max_bin=None,

max_cat_threshold=None,

max_cat_to_onehot=None,
                                                        max_delta_step=None,
                                                        max_depth=None,
                                                        max_leaves=None,

min_child_weight=None,
                                                        missing=nan,

monotone_constraints=None,
                                                        multi_strategy=None,
                                                        n_estimators=None,
                                                        n_jobs=None,

num_parallel_tree=None, ...))
[-0.16614882 -0.16376356 -0.17207427 -0.17102108 -0.16457591]
TransformedTargetRegressor(func=<ufunc 'log'>, inverse_func=<ufunc
'exp'>,
                           regressor=XGBRegressor(base_score=None,
booster=None,
                                                        callbacks=None,

colsample_bylevel=None,

colsample_bynode=None,

colsample_bytree=None,
                                                        device=None,

early_stopping_rounds=None,

enable_categorical=False,
                                                        eval_metric=None,
                                                        feature_types=None,

feature_weights=None,
                                                        gamma=None,
```

```
grow_policy=None,

importance_type=None,

interaction_constraints=None,
                                                        learning_rate=None,
                                                        max_bin=None,

max_cat_threshold=None,

max_cat_to_onehot=None,
                                                        max_delta_step=None,
                                                        max_depth=None,
                                                        max_leaves=None,

min_child_weight=None,
                                                        missing=nan,

monotone_constraints=None,
                                                        multi_strategy=None,
                                                        n_estimators=None,
                                                        n_jobs=None,

num_parallel_tree=None, ...))
```

```
## Evaluasi hasil dari 5 kandidat algoritma training
score_training = pd.DataFrame({
    'Model': ['Training Model Linear Regression', 'Training Model KNN
Regressor', 'Training Model DecisionTree Regressor', 'Training Model
RandomForest Regressor', 'Training Model XGBoost Regressor'],
    'Mean_MAE': nilai_mean_mae,
    'Std_MAE': nilai_std_mae,
    'Mean_MAPE': nilai_mean_mape,
    'Std_MAPE': nilai_std_mape
})
print('Skor Training :')
score_training
```

```
Skor Training :

                                   Model      Mean_MAE      Std_MAE  \
0       Training Model Linear Regression -41610.294376  1021.166612
1           Training Model KNN Regressor -36084.696345   632.965432
2  Training Model DecisionTree Regressor -42261.929044  1566.002702
3  Training Model RandomForest Regressor -30348.070977   351.956875
4       Training Model XGBoost Regressor -29285.679935   269.441948

   Mean_MAPE  Std_MAPE
0  -0.239221  0.004294
1  -0.201876  0.005468
2  -0.245752  0.005314
```

```
3  -0.172244  0.002248
4  -0.167517  0.003396
```

Pemilihan Modeling Machine Learning

```
## Melakukan testing pada base model
models = {
    'Testing Linear Regression': LinearRegression(),
    'Testing KNN': KNeighborsRegressor(),
    'Testing Decision Tree Regressor':
DecisionTreeRegressor(random_state=42),
    'Testing RandomForest': RandomForestRegressor(random_state=42),
    'Testing XGBoost': XGBRegressor(random_state=42)
}

score_mae = []
score_mape = []

## Melakukan prediksi pada test set
for i in models:

    model = Pipeline([
        ('preprocessing', transformer),
        ('model', models[i])
        ])

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    score_mae.append(mean_absolute_error(y_test, y_pred))
    score_mape.append(mean_absolute_percentage_error(y_test, y_pred))

score_before_tuning = pd.DataFrame({'MAE': score_mae, 'MAPE':
score_mape}, index=models.keys())

## Evaluasi hasil dari 5 kandidat algoritma testing
print('Hasil Test Set (MAE dalam bentuk US Dollar, MAPE dalam bentuk
persentase): ')
score_before_tuning
```

```
Hasil Test Set (MAE dalam bentuk US Dollar, MAPE dalam bentuk
persentase):
```

```
                                          MAE      MAPE
Testing Linear Regression        42984.752594  0.267082
Testing KNN                      36684.157799  0.211249
Testing Decision Tree Regressor  42409.859677  0.251364
Testing RandomForest             30915.685417  0.184892
Testing XGBoost                  29607.674677  0.177392
```

```
## Kedalaman pohon
max_depth = list(np.arange(1, 11)) ## 1-10
```

```python
## Learning rate
learning_rate = list(np.arange(1, 100)/100)

## Jumlah pohon
n_estimators = list(np.arange(100, 201))

## Jumlah baris tiap pohon (% dari total baris train set)
subsample = list(np.arange(2, 10)/10)

## Gamma (min_impurity_decrease)
gamma = list(np.arange(1, 11))

## Jumlah feature yang digunakan untuk tiap pohon (% dari total kolom
train set)
colsample_bytree = list(np.arange(1, 10)/10)

## Alpha (regularization)
reg_alpha = list(np.logspace(-3, 1, 10))


## Hyperparam space XGboost
hyperparam_space_xgb = {
    'model__max_depth': max_depth,
    'model__learning_rate': learning_rate,
    'model__n_estimators': n_estimators,
    'model__subsample': subsample,
    'model__gamma': gamma,
    'model__colsample_bytree': colsample_bytree,
    'model__reg_alpha': reg_alpha
}

## Benchmark model dengan hyperparameter tuning yang dipilih :
xgb = XGBRegressor(random_state=42, verbosity=0)
## verbosity = 0 artinya tidak menampilkan informasi progress
hyperparameter tuning yang di running oleh program saat di
train_test_split
## verbosity = 1 artinya  menampilkan informasi progress
hyperparameter tuning (seperti epoch berapa, step berapa) yang di
running oleh program saat di train_test_split
## verbosity = 2 artinya  menampilkan informasi progress
hyperparameter tuning (1/64 Epoch) yang di running oleh program saat
di train_test_split


## Membuat algorithm chains
estimator_xgb = Pipeline([
        ('preprocessing', transformer),
        ('model', xgb)
        ])
```

```
crossval = KFold(n_splits=5, shuffle=True, random_state=42) ## n_split
5 kombinasi, #shuffle untuk mengacak -> dibagi 5 berdasarkan n_splits

## Hyperparameter tuning
hyperparameter_tuning_xgb = RandomizedSearchCV(
    estimator_xgb,
    param_distributions = hyperparam_space_xgb,
    n_iter = 50,
    cv = crossval,
    scoring = ['neg_mean_absolute_error',
'neg_mean_absolute_percentage_error'],
    n_jobs = -1,
    refit = 'neg_mean_absolute_percentage_error',
    random_state = 42
)

## Fitting data training untuk mencari parameter terbaik
hyperparameter_tuning_xgb.fit(X_train, y_train)
```

```
RandomizedSearchCV(cv=KFold(n_splits=5, random_state=42,
shuffle=True),
                   estimator=Pipeline(steps=[('preprocessing',

ColumnTransformer(remainder='passthrough',

transformers=[('cat',

Pipeline(steps=[('imputer',

SimpleImputer(strategy='most_frequent')),

('onehot',

OneHotEncoder(drop='first',

handle_unknown='ignore'))]),

['ocean_proximity']),

('num',

Pipeline(steps...

np.float64(1.2915496650148828),

np.float64(3.593813663804626),

np.float64(10.0)],
                                          'model__subsample':
[np.float64(0.2),
```

```
np.float64(0.3),

np.float64(0.4),

np.float64(0.5),

np.float64(0.6),

np.float64(0.7),

np.float64(0.8),

np.float64(0.9)]},
                    random_state=42,
refit='neg_mean_absolute_percentage_error',
                    scoring=['neg_mean_absolute_error',
                             'neg_mean_absolute_percentage_error'])

## Melihat hasil tuning dalam bentuk dataframe. Diurutkan berdasarkan
MAE
hasil_error =
pd.DataFrame(hyperparameter_tuning_xgb.cv_results_).sort_values(by=['r
ank_test_neg_mean_absolute_error',
'rank_test_neg_mean_absolute_percentage_error']).head()
hasil_error
```

```
    mean_fit_time  std_fit_time  mean_score_time  std_score_time  \
39       3.356892      0.062760         0.062496        0.003325
26       0.948067      0.025465         0.031707        0.002219
9        0.564028      0.015962         0.031882        0.003642
18       1.918149      0.074257         0.047074        0.002048
31       0.669501      0.036809         0.032495        0.002302

    param_model__subsample  param_model__reg_alpha
param_model__n_estimators  \
39                     0.6                0.059948
141
26                     0.4                1.291550
100
9                      0.9                0.059948
105
18                     0.2                0.059948
193
31                     0.9               10.000000
176

    param_model__max_depth  param_model__learning_rate
param_model__gamma  \
39                      10                        0.03
10
```

```
26                          7                          0.06
1
9                           5                          0.38
3
18                          8                          0.08
4
31                          3                          0.48
3

    ...  std_test_neg_mean_absolute_error
rank_test_neg_mean_absolute_error  \
39  ...                          431.145820
1
26  ...                          343.267812
2
9   ...                          505.969504
3
18  ...                          415.434378
4
31  ...                          624.029175
5

    split0_test_neg_mean_absolute_percentage_error  \
39                                        -0.179619
26                                        -0.183207
9                                         -0.182786
18                                        -0.183093
31                                        -0.188167

    split1_test_neg_mean_absolute_percentage_error  \
39                                        -0.175652
26                                        -0.176592
9                                         -0.181657
18                                        -0.182291
31                                        -0.189201

    split2_test_neg_mean_absolute_percentage_error  \
39                                        -0.186637
26                                        -0.184868
9                                         -0.192572
18                                        -0.190634
31                                        -0.203440

    split3_test_neg_mean_absolute_percentage_error  \
39                                        -0.184403
26                                        -0.184081
9                                         -0.187075
18                                        -0.187754
31                                        -0.193855
```

```
    split4_test_neg_mean_absolute_percentage_error  \
39                                       -0.174188
26                                       -0.179171
9                                        -0.177343
18                                       -0.181841
31                                       -0.185801

    mean_test_neg_mean_absolute_percentage_error  \
39                                     -0.180100
26                                     -0.181584
9                                      -0.184286
18                                     -0.185123
31                                     -0.192093

    std_test_neg_mean_absolute_percentage_error  \
39                                     0.004821
26                                     0.003175
9                                      0.005173
18                                     0.003470
31                                     0.006249

    rank_test_neg_mean_absolute_percentage_error
39                                             1
26                                             2
9                                              3
18                                             4
31                                             6

[5 rows x 28 columns]
```

```python
## Cek skor terbaik dan params terbaik
print('XGBoost')
print('Skor Terbaik :', hyperparameter_tuning_xgb.best_score_)
print('Parameter Terbaik :', hyperparameter_tuning_xgb.best_params_)
```

```
XGBoost
Skor Terbaik : -0.18009980504704526
Parameter Terbaik : {'model__subsample': np.float64(0.6),
'model__reg_alpha': np.float64(0.05994842503189409),
'model__n_estimators': np.int64(141), 'model__max_depth':
np.int64(10), 'model__learning_rate': np.float64(0.03),
'model__gamma': np.int64(10), 'model__colsample_bytree':
np.float64(0.6)}
```

```python
## Menggunakan Model XGBoost
model = {'XGB': XGBRegressor(random_state=42)}

## Define model terhadap estimator terbaik
xgb_tuning = hyperparameter_tuning_xgb.best_estimator_

## Fitting model
```

```
xgb_tuning.fit(X_train, y_train)

## Predict test set
y_pred_xgb_tuning = xgb_tuning.predict(X_test)

## Simpan nilai metrics MAE
mae_xgb_tuning = mean_absolute_error(y_test, y_pred_xgb_tuning)

## Simpan nilai metrics MAPE
mape_xgb_tuning = mean_absolute_percentage_error(y_test,
y_pred_xgb_tuning)

score_after_tuning = pd.DataFrame({'MAE': mae_xgb_tuning, 'MAPE':
mape_xgb_tuning}, index=model.keys())
print("Nilai MAE Algorimat XGB setelah Tunning : ")
score_after_tuning
```

```
Nilai MAE Algorimat XGB setelah Tunning :
```

```
             MAE     MAPE
XGB  29753.85603  0.18259
```

```
## Sebelum hyperparameter tuning
print("TABEL MODEL XGB SEBELUM TUNING : ")
pd.DataFrame(score_before_tuning.loc['Testing XGBoost']).T
```

```
TABEL MODEL XGB SEBELUM TUNING :
```

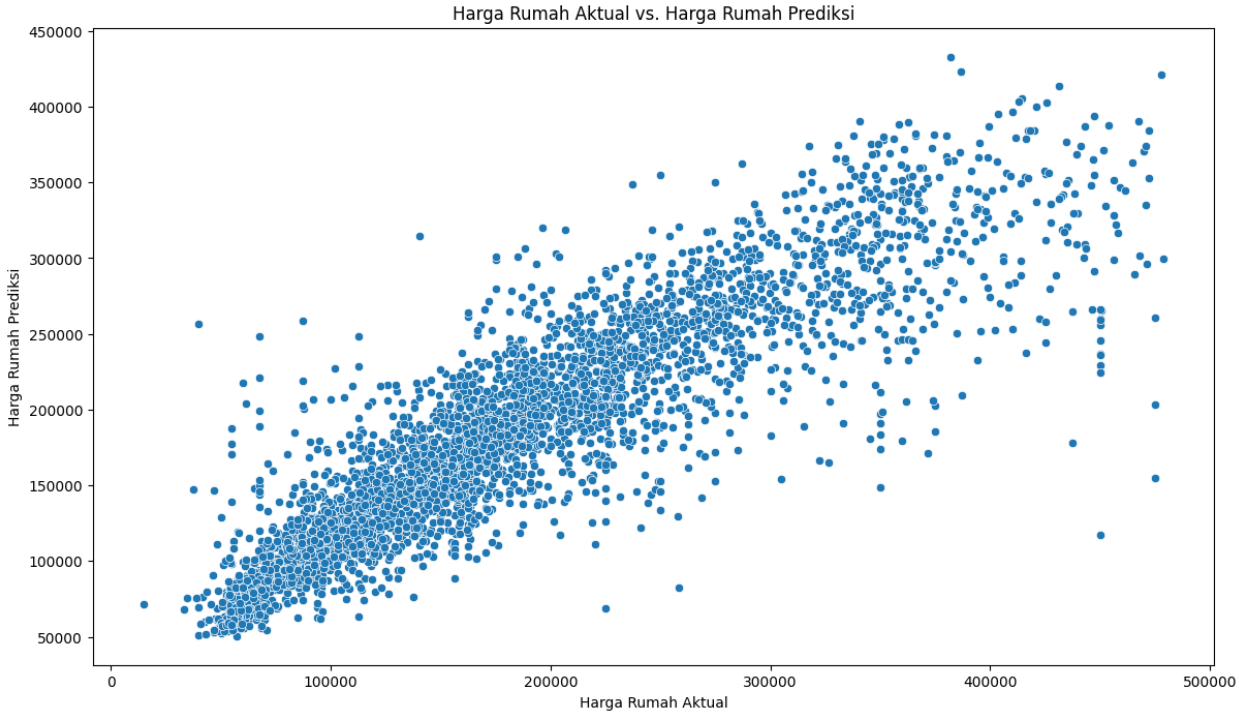
```
                          MAE      MAPE
Testing XGBoost  29607.674677  0.177392
```

```
## Setelah hyperparameter tuning
print("TABEL MODEL XGB SETELAH TUNING : ")
score_after_tuning
```

```
TABEL MODEL XGB SETELAH TUNING :
```

```
             MAE     MAPE
XGB  29753.85603  0.18259
```

```
## Visualisasi data Aktual dengan Data Predict
plt.figure(figsize=(14, 8))
plot = sns.scatterplot(x=y_test, y=y_pred_xgb_tuning).set(title='Harga
Rumah Aktual vs. Harga Rumah Prediksi', xlabel='Harga Rumah Aktual',
ylabel='Harga Rumah Prediksi');
```




```
## Menggabungkan kembali feature yang tidak masuk dalam pipelilne
dengan tujuan untuk melihat pengaruh variabel x ke variabel y
features = (
    list(transformer.transformers_[0][1][0].get_feature_names_out()) +
    transformer.transformers_[1][2] +
    ['longitude','latitude']
)

features
```

```
['ocean_proximity',
 'housing_median_age',
 'total_rooms',
 'total_bedrooms',
 'population',
 'households',
 'median_income',
 'longitude',
 'latitude']
```

```
##  Hasil yang tidak masuk dalam pipeline dimasukan dalam sebuah
variabel dan dijadikan data frame
features = transformer.get_feature_names_out()

feature_importances = pd.DataFrame(
    data=xgb_tuning[1].feature_importances_,
    index=features,
    columns=["coef"]
```

```
)

feature_importances.sort_values(by="coef", ascending=False,
inplace=True)

feature_importances
```

```
                                      coef
cat__ocean_proximity_INLAND       0.433901
remainder__ocean_proximity_temp   0.302736
num__median_income                0.094643
remainder__longitude              0.035464
remainder__latitude               0.030368
cat__ocean_proximity_NEAR BAY     0.016191
cat__ocean_proximity_NEAR OCEAN   0.015506
num__population                   0.015274
num__total_rooms                  0.013998
num__households                   0.012893
num__total_bedrooms               0.010594
num__housing_median_age           0.009845
cat__ocean_proximity_ISLAND       0.008589
```

3.2 Melakukan Proses Review Pemodelan

- Menilai kesesuaian proses permodelan
- Menilai kualitas proses pemodelan

Sebelum pakai Machine Learning

```
## Membuat dataframe baru yang bersumber dari dataframe df
df_perbandingan = df.copy()

## Menampilkan kolom lokasi perumahan dan harga rumah aktual
df_perbandingan[['latitude','longitude','median_house_value']].head()
```

```
   latitude  longitude  median_house_value
1     37.77    -122.21            100000.0
2     33.87    -118.04            285800.0
3     34.06    -118.28            175000.0
4     36.73    -119.81             59200.0
5     33.80    -117.79            283200.0
```

Setelah Pakai Machine Learning

```
## Membuat kolom Harga_Prediksi_Rumah dengan menggunakan data hasil
prediksi xgboost tuning
df_perbandingan['Harga_Prediksi_Rumah'] =
(xgb_tuning.predict(X)).round()
```

```
## Membuat kolom Selisih_Harga , selisih harga ini antara harga aktual dengan dengan harga prediksi
df_perbandingan['Selisih_Harga'] = abs(df_perbandingan['median_house_value']-df_perbandingan['Harga_Prediksi_Rumah'])

## Menampilkan kolom harga aktual, kolom harga prediksi, kolom selisih harga
df_perbandingan[['latitude','longitude','median_house_value','Harga_Prediksi_Rumah','Selisih_Harga']].head()
```

```
   latitude  longitude  median_house_value  Harga_Prediksi_Rumah  \
1     37.77    -122.21            100000.0               99881.0
2     33.87    -118.04            285800.0              293138.0
3     34.06    -118.28            175000.0              187743.0
4     36.73    -119.81             59200.0               63638.0
5     33.80    -117.79            283200.0              299758.0

   Selisih_Harga
1          119.0
2         7338.0
3        12743.0
4         4438.0
5        16558.0
```

```
## Membuat variabel baru yang berisikan sampel khusus untuk memperlihatkan perbandingan harga aktual dengan harga prediksi dan selisihnya
## Index yang digunakan adalah index paling atas dari data agar memudahkan perbandingan.
df_perbandingan_n = df_perbandingan.iloc[:2,9:13]
df_perbandingan_n
```

```
   median_house_value  ocean_proximity_temp  Harga_Prediksi_Rumah  \
1            100000.0                     3               99881.0
2            285800.0                     0              293138.0

   Selisih_Harga
1          119.0
2         7338.0
```

BAB 4 : KESIMPULAN DAN SARAN

- 4.1 Kesimpulan
  - Berdasarkan hasil pengembangan dan evaluasi model prediksi harga rumah, diperoleh bahwa algoritma XGBoost memberikan performa terbaik dibandingkan model lainnya seperti Linear Regression, KNeighborsRegressor, Decision Tree, dan Random Forest. Hal ini terlihat dari nilai MAE dan MAPE yang paling rendah baik pada tahap training maupun testing sehingga model tersebut dipilih sebagai model akhir dan dilakukan proses hyperparameter tuning untuk memaksimalkan

performanya. Setelah dilakukan tuning, model menghasilkan nilai MAE sekitar 28,293 USD dengan MAPE sebesar 0.168612 (≈16%), yang menunjukkan bahwa rata-rata selisih antara harga prediksi dan harga aktual berada pada kisaran sekitar 28 ribu dolar atau sekitar 16% dari harga sebenarnya. Hasil analisis juga menunjukkan bahwa fitur median_income dan ocean_proximity memiliki pengaruh besar dalam menentukan harga rumah karena kedua fitur tersebut berkaitan dengan kondisi ekonomi wilayah dan kedekatan lokasi terhadap pantai yang secara umum memengaruhi nilai properti. Meskipun model mampu memberikan prediksi yang cukup akurat, terdapat beberapa batasan pada proyek ini, seperti penggunaan fitur housing_median_age yang hanya mencakup umur rumah hingga 50 tahun serta batas harga rumah maksimum sekitar 462,200 USD sesuai dengan karakteristik dataset yang digunakan. Secara keseluruhan, penerapan machine learning ini mampu membantu agen properti dalam memperkirakan harga rumah secara lebih objektif dan berbasis data sehingga dapat mendukung pengambilan keputusan harga yang lebih tepat di pasar properti.

  - Selain itu, hasil proyek ini juga menunjukkan bahwa penerapan model prediksi harga rumah mampu mendukung pencapaian objektif bisnis yang telah ditetapkan. Model ini dapat membantu agen properti menentukan harga rumah yang lebih akurat dan kompetitif berdasarkan analisis data sehingga harga yang ditentukan tidak terlalu tinggi maupun terlalu rendah dibandingkan kondisi pasar. Dengan adanya estimasi harga yang lebih realistis, peluang rumah untuk terjual juga dapat meningkat karena harga yang ditawarkan lebih sesuai dengan ekspektasi calon pembeli. Di sisi lain, model ini juga membantu mengurangi risiko rumah tidak terjual dalam waktu lama akibat penetapan harga yang kurang tepat. Prediksi harga yang dihasilkan dapat dijadikan sebagai rekomendasi berbasis data bagi pemilik rumah, sehingga agen properti dapat memberikan penjelasan yang lebih objektif dan profesional kepada klien. Dengan demikian, penggunaan model machine learning ini tidak hanya meningkatkan efisiensi dalam menentukan harga properti, tetapi juga dapat meningkatkan kepercayaan klien terhadap agen properti karena keputusan harga didasarkan pada analisis data yang sistematis dan terukur.

- 4.2 Saran Beberapa saran yang dapat dilakukan untuk pengembangan penelitian atau proyek selanjutnya antara lain:
  - Menambahkan lebih banyak fitur yang relevan seperti luas bangunan, jumlah kamar tidur, jumlah kamar mandi, fasilitas lingkungan, atau jarak ke pusat kota agar model dapat menangkap faktor yang lebih lengkap dalam menentukan harga rumah.
  - Menggunakan dataset yang lebih besar dan lebih bervariasi agar model dapat mempelajari pola harga rumah dari berbagai kondisi wilayah dan menghasilkan prediksi yang lebih akurat.
  - Melakukan eksplorasi algoritma machine learning lainnya seperti model boosting atau deep learning untuk membandingkan performa dan mencari model dengan akurasi yang lebih tinggi.
  - Melakukan tuning hyperparameter yang lebih mendalam serta menggunakan teknik validasi tambahan agar performa model semakin optimal dan stabil.

- Mengembangkan sistem prediksi dalam bentuk aplikasi atau dashboard sehingga agen properti dapat dengan mudah memasukkan data rumah dan langsung memperoleh rekomendasi harga secara otomatis.